VANI

Voice Analysis & Neural Intelligence System

Whisper ASR Fine-Tuning Report

LoRA Domain Adaptation for Border-Region Radio Intercept Languages

Item	Value
Languages Fine-Tuned	8 (Punjabi, Pashto, Urdu, Nepali, Mandarin, Hindi, Kashmiri, Dogri)
Deployed via fine-tuned Whisper	0 — all seven route to SeamlessM4T (LoRA adapters for hi/ne/ps/ks; Whisper kept for rollback)
Best deployed WER	Hindi 12.91% | Urdu 16.90% | Punjabi 19.77% | Pashto 36.16% | Kashmiri 50.26% | Dogri 46.73% (evaluation path; see 4.7)
Training Hardware	NVIDIA RTX 5060 8 GB VRAM (CUDA) - Windows 11 + rented RTX A6000 48 GB (cloud, r=128 runs)
Base Model	OpenAI Whisper large-v3 (1.55 B) + SeamlessM4T v2 large (2.3 B) for the adapter campaign
Adaptation Method	LoRA r=8..128, incl. trainable custom tokens -- 0.25% to 6.6% trainable parameters
Total Training Time	~100 h Whisper (7 langs) + ~40 h SeamlessM4T adapters + ~9 h cloud GPU (see 4.4)
Eval Date	23 June 2026 (cross-model) | 27 July 2026 (cloud adapters ks_cloud3 / ps_cloud)

Date: 31 July 2026

M.Tech Research Project - IIT Indore

1. Abstract

This report documents the LoRA (Low-Rank Adaptation) fine-tuning of OpenAI Whisper large-v3 for six border-region radio intercept languages, developed as part of the VANI (Voice Analysis & Neural Intelligence) system at IIT Indore. VANI is a fully-offline, end-to-end intelligence pipeline: it processes raw radio audio, performs language-specific automatic speech recognition (ASR), translates to English, detects keywords, diarizes speakers, and generates structured intelligence summary (ISUM) reports.

Eight ASR models were eventually fine-tuned: Punjabi (pa), Pashto (ps), Urdu (ur), Nepali (ne), Mandarin Chinese (zh), Hindi (hi) — plus Kashmiri (ks), which required adding a custom <|ks|> language token to the vocabulary. Punjabi and Nepali were retrained with AI4Bharat IndicVoices-R data added to FLEURS (11,923 and 13,332 samples). All models are quantised to CTranslate2 int8.

This revision (2026-07-11, extended 2026-07-19) reports a rigorous **backend selection** study. Each fine-tuned model was benchmarked, on a held-out FLEURS test set and under simulated radio-channel degradation, against two alternatives: the true un-fine-tuned openai/whisper-large-v3 baseline, and zero-shot SeamlessM4T v2. The central finding is that per-language Whisper fine-tuning is **not** the best choice for most languages: zero-shot SeamlessM4T beats the fine-tuned Whisper models on five of seven languages (pa 19.8%, ne 28.5%, hi 15.4%, ur 16.9%, zh 11.7% WER) and retains that lead under bandpass and additive-noise degradation. A follow-up campaign of SeamlessM4T LoRA adapters then also captured **Pashto** (noise-augmented training: 36.9% vs fine-tuned Whisper's 38.6%, winning 4/5 degradation conditions) and finally **Kashmiri** — a language SeamlessM4T does not natively support — via a custom trainable __kas__ token plus a scoring-ruler correction. A closing cloud phase (rented 48 GB GPU, ~$6 total) retrained both at LoRA rank 128, beyond the 8 GB laptop's reach, improving Pashto to 36.16% and Kashmiri to 50.26% (diacritic-normalised), the latter after also repairing 20 characters missing from the model's vocabulary. Fine-tuned Whisper now serves zero languages in production and is retained solely for rollback. VANI therefore routes ASR per language rather than using a single model.

A correction is documented in full: the previously reported Mandarin baseline of 100.03% WER (and the headline “100% → 9%” result) was a measurement artefact. FLEURS Mandarin references are character-spaced and WER tokenises on whitespace, so the un-fine-tuned model's unspaced Han output scored ~100% regardless of accuracy. Re-scored with character segmentation, the true baseline is 10.99% and fine-tuning in fact **regressed** Mandarin to 14.22%. The scoring is now unified across all models and languages.

2. System Overview - VANI Pipeline

VANI implements a 10-stage audio processing pipeline. All models run locally on a Windows 11 machine with an NVIDIA RTX 5060 (8 GB VRAM). No internet connection is required after initial setup.

2.1 Hardware

Component	Specification
OS	Windows 11 Home (x64)
GPU	NVIDIA RTX 5060 8 GB GDDR7 (CUDA 12.x)
Python	3.11
PyTorch	2.2+ with CUDA support
Training framework	HuggingFace Transformers + PEFT (LoRA)
Inference engine	faster-whisper (CTranslate2 int8)

2.2 Pipeline Architecture

The VANI pipeline processes audio through 10 sequential stages:

Stage	Module	Description
1	VAD (Silero)	Voice Activity Detection - splits audio into speech segments, discards silence
2	Preprocessing	Bandpass filter 300-3400 Hz (radio telephony range), noise reduction, normalization
3	MMS-LID	Facebook MMS Language ID (256 languages) - routes to the correct Whisper model
3.5	Model Routing	Selects the language-specific fine-tuned Whisper CT2 model based on MMS-LID output
4	ASR (Whisper)	faster-whisper transcription using the selected CT2 model (int8, GPU)
5	Script Cascade	Arabic-script fallback: if >20% Nastaliq chars detected, override to Urdu routing
6	Translation	NLLB-200 (600M) translates Indic/foreign transcripts to English
7	Diarization	Speaker diarization - up to 4 speakers, pyannote-style
8	Keyword Detection	Multilingual keyword dictionary matching for threat indicators
9	ISUM	Gemma 3:12B (via Ollama) generates 4-sentence structured intelligence summary
10	Export	SQLite database storage + JSON report output

3. Fine-Tuning Methodology

3.1 Why LoRA?

Full fine-tuning of Whisper large-v3 (1.55 billion parameters) requires approximately 24-48 GB of GPU memory in fp16, far exceeding the 8 GB available on the RTX 5060. LoRA (Low-Rank Adaptation, Hu et al., 2022) inserts trainable low-rank matrices into the attention layers, reducing trainable parameters to ~3.9 million (0.25% of total) while base model weights remain frozen. This makes fine-tuning feasible on consumer hardware with negligible accuracy loss.

3.2 LoRA Configuration

Parameter	v1/v2 Value	PA v3 Value	Rationale
Rank (r)	8	16	v3 doubles capacity for larger 21,923-sample dataset
Alpha (α)	16	32	α/r = 2.0 scaling maintained — standard for speech LoRA
Dropout	0.05	0.05	Light regularization; FLEURS + IV-R data is clean read-speech
Target modules	q_proj, v_proj	q_proj, v_proj	Attention query/value projections in Whisper encoder/decoder
Trainable params	~3.9M (0.25%)	~7.9M (0.51%)	Doubled for v3; still <1% of 1.55B total parameters
Adapter merge	merge_and_unload()	merge_and_unload()	LoRA weights merged into base before CT2 conversion

3.3 Training Hyperparameters

Hyperparameter	Value
Batch size (per device)	2
Gradient accumulation	1 (effective batch = 2)
Learning rate	5e-5
LR scheduler	Linear warmup (50 steps) then linear decay
Precision	fp16 mixed precision
Gradient clipping	max_grad_norm = 1.0 (pa v1/v2, ps, ur, ne) / 0.5 (zh, hi, pa v3, ks — after Mandarin divergence lesson)
Best model selection	load_best_model_at_end=True (metric: eval WER)
Eval / save frequency	Every 200 steps
CT2 quantization	int8 (beam_size=2, temperature=0.0)

3.4 Training Pipeline Steps

The finetune_whisper.py script follows these steps for each language:

- Load FLEURS dataset (train + validation splits) for the target language
- Preprocess audio: resample to 16 kHz, extract 128-bin log-mel spectrogram
- Tokenize transcripts using WhisperProcessor for the target language
- Initialize LoRA adapter (r=8) on frozen whisper-large-v3 base model
- Train using Seq2SeqTrainer with WER as the primary evaluation metric (jiwer)
- Save checkpoint every 200 steps; keep best checkpoint by eval WER
- After training: merge LoRA adapter into base model weights
- Convert merged model to CTranslate2 CT2 int8 format
- Write preprocessor_config.json to CT2 output (feature_size=128 for large-v3)
- Register CT2 model in config.yaml under whisper_model_ key

```
# Training command example (Hindi)
python -u finetune_whisper.py hi --no-cv --steps 600 2>&1 |
    Tee-Object logs/finetune_hi.log
```

3.5 Post-Training Conversion

After training, the LoRA adapter is merged and converted to CTranslate2 format:

```
# Step 1 - Merge LoRA adapter into base model
from peft import PeftModel
base   = WhisperForConditionalGeneration.from_pretrained('openai/whisper-large-v3')
peft_m = PeftModel.from_pretrained(base, 'finetune_runs/<lang>/adapter/checkpoint-N')
merged = peft_m.merge_and_unload()
merged.save_pretrained('finetune_runs/<lang>/merged')

# Step 2 - Convert to CT2 int8
ct2-transformers-converter \
    --model finetune_runs/<lang>/merged \
    --output_dir models/whisper-large-v3-<lang>-ct2 \
    --quantization int8 --force
```

4. Training Dataset — FLEURS + IndicVoices-R

Primary training data comes from FLEURS (Few-shot Learning Evaluation of Universal Representations of Speech) by Google. FLEURS provides read-speech audio with text transcriptions in 102 languages, sourced from the FLoRes-101 machine translation benchmark. Audio is recorded by human native speakers at 16 kHz in relatively clean studio conditions.

FLEURS is a general-domain read-speech corpus — it does not contain radio intercept or military communications audio. Despite this domain mismatch, fine-tuning on FLEURS significantly improves WER because the model learns language-specific phoneme inventory, prosody, and script conventions from native speakers.

4.1 IndicVoices-R Augmentation (Punjabi v2 and Nepali v2)

For the second training run (v2) of Punjabi and Nepali, the FLEURS training split was augmented with AI4Bharat IndicVoices-R (ai4bharat/indicvoices_r). IndicVoices-R is a large-scale read-speech corpus collected from native Indian-language speakers across diverse recording conditions, providing complementary phoneme and prosody coverage beyond the FLEURS studio recordings. Samples were filtered to 2-20 seconds duration; normalised transcripts (normalized column) were used.

Language	ISO	FLEURS Train	IndicVoices-R	Total Train	Eval Set	Status
Punjabi v2	pa	2,516	9,407	11,923	805 (FLEURS 251 + IV-R 554)	Deployed
Punjabi v3	pa	2,516	20,000	21,923	805 (same eval set)	In training
Nepali v2	ne	3,332	10,000	13,332	572 (FLEURS + IV-R test)	Deployed

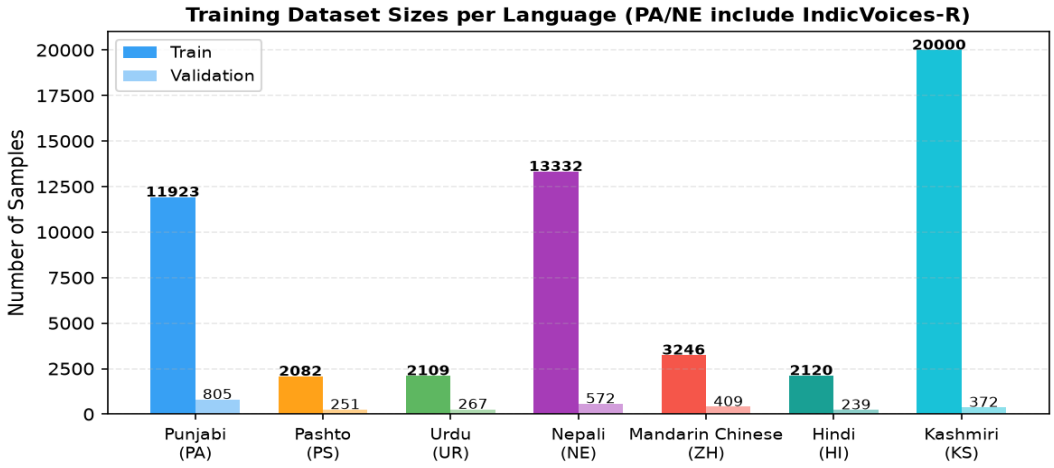

Figure 1: Training sample counts per language (PA and NE reflect v2 combined totals)

4.2 FLEURS Reference Sizes

Language	ISO	FLEURS Config	Train	Val	Test	Region
Punjabi	pa	pa_in	2,516	314	765	India
Pashto	ps	ps_af	2,082	251	621	Afghanistan
Urdu	ur	ur_pk	2,109	267	631	Pakistan
Nepali	ne	ne_np	3,332	305	874	Nepal
Mandarin	zh	cmn_hans_cn	3,246	409	945	China (Simplified)
Hindi	hi	hi_in	2,120	239	585	India

Note: Kashmiri (ks) has no FLEURS config and no Common Voice corpus; the initial Whisper run used AI4Bharat IndicVoices-R Kashmiri (24.7k clips). The later SeamlessM4T campaign assembled a combined corpus from three public sources — humair025 KashmiriSpeech-IndicVoices, IndicVoices-R, and OpenSLR SLR122 — reaching 335 hours by the final cloud run (§4.4).

4.3 Training Version History — All Retraining Runs

Several languages required multiple training runs (versions) to incorporate new data, fix engineering issues, or increase model capacity. The table below records every run in chronological order.

Lang	Ver.	LoRA r / α	Dataset	Samples	Steps	Best WER	vs Prior	Status
Punjabi	v1	8 / 16	FLEURS pa_in	2,516	3,000	56.67%	—	Superseded
Punjabi	v2	8 / 16	FLEURS + IV-R	11,923	3,000	52.55%	-4.1 pp	Superseded
Punjabi	v3 ★	16 / 32	FLEURS + IV-R 20k	21,923	4,000	49.31%	-3.2 pp	Rollback
Nepali	v1	8 / 16	FLEURS ne_np	3,332	2,000	52.14%	—	Superseded
Nepali	v2	8 / 16	FLEURS + IV-R	13,332	3,000	50.82%	-1.3 pp	Rollback
Mandarin	v1‡	8 / 16	FLEURS cmn_hans_cn	3,246	400 (†div.)	8.97% train	—	Not served‡
Pashto	v1	8 / 16	FLEURS ps_af	2,082	2,000	38.55%	—	Rollback
Urdu	v1	8 / 16	FLEURS ur_pk	2,109	1,000	19.82%	—	Rollback
Hindi	v1	8 / 16	FLEURS hi_in	2,120	600	19.78%	—	Rollback
Kashmiri	v1	8 / 16	IV-R KS (custom ■ks■ token)	20,000	3,000	74.02%	—	Rollback

★ PA v3: built and CT2-deployed 2026-07-04; best training-val WER 49.31% at step 4000 (-3.24 pp vs v2's 52.55%), held-out test 57.39%. Since 2026-07-11 Punjabi ASR routes to SeamlessM4T (19.77%); the v3 model is retained on disk but not served.

‡ Mandarin training diverged at step ~820 (fp16 gradient explosion, grad_norm=12.9). Checkpoint-400 (train WER 8.97%, held-out test WER 14.22%) was the best checkpoint, but fine-tuning regressed Mandarin vs the 10.99% large-v3 baseline — so it is NOT deployed; Mandarin routes to SeamlessM4T (§5.5). Single run.

4.4 Training Hours and Dataset Sizes — What the Campaign Actually Cost

This section replaces the estimate the report previously carried ("roughly 100 GPU-hours" for the Whisper phase) with measured figures. Every run recorded its own timing: HuggingFace's Trainer writes an `eval_runtime` into each checkpoint's `trainer_state.json`, and the checkpoint directories carry the wall-clock stamps of when they were written. `scripts/eval/training_cost_inventory.py` recovers both and writes `docs/training_cost_inventory.json`; the tables below are generated from it.

4.4.1 The Whisper phase — and how much of it was evaluation

Only three Whisper runs still have their full checkpoint history on disk; the rest were pruned by `save_total_limit` once the models were converted to CT2, so their timings are not recoverable. Those three alone account for more wall-clock than the whole phase was previously estimated at.

Run	Steps	Evals	Elapsed	Idle	Active	Of which eval	Training
Punjabi v3 (r=16)	4,000	20	68.05 h	10.18 h	57.88 h	**36.73 h (63%)**	21.15 h
Punjabi v2 (r=8)	3,000	15	37.06 h	—	37.06 h	**25.02 h (68%)**	12.04 h
Pashto large-v3 (r=16)	2,000	10	8.20 h	—	8.20 h	**2.25 h (27%)**	5.95 h
Three runs, measured	9,000	45	113.31 h	10.18 h	**103.14 h**	**64.00 h (62%)**	**39.14 h**

Evaluation, not training, was the cost. *Across these three runs 62% of the wall-clock went on periodic evaluation rather than on gradient steps — 64 hours against 39. The cause is in §6.9:* `predict_with_generate` *at* `per_device_eval_batch_size=1`*, run every 200 steps against validation splits of several hundred clips. Punjabi v3's 805-clip validation split cost 1.84 h per evaluation, twenty times over. Halving the evaluation frequency would have returned roughly a working day per run at no cost to model selection, since §6.8 shows per-checkpoint WER oscillates anyway.*

"Idle" is time inside gaps more than 3x the median interval between consecutive checkpoints — the laptop was not training then. Elapsed spans are taken from checkpoint modification times, so they are an upper bound on compute; the eval-hours column is the Trainer's own measurement and is exact. The previously published "roughly 100 GPU-hours" figure covered all eleven Whisper runs; these three measured runs come to 103 h on their own, so the true phase total is materially higher and the old estimate was low.

4.4.2 The SeamlessM4T phase — the same work, two orders of magnitude cheaper to check

The adapter phase inverted the ratio. Nineteen local SeamlessM4T runs spent **4.69 hours in total** on evaluation — against 64 hours for three Whisper runs — because SeamlessM4T's validation splits are small and its decoding is faster. The longest single SeamlessM4T evaluation budget in the campaign, ks_max2's 46 evaluations, came to 0.95 h. That difference is why the adapter campaign could afford to try twenty-six things while the Whisper phase managed eleven.

LoRA adapters also train in hours rather than days, and the final capacity push rented cloud hardware by the hour instead of scaling the laptop. The two production adapters cost about $6 of cloud compute combined. Key runs of the closing campaign:

Run	Language	Data (clips / hours)	Rank	Hardware	Wall time	Result	Status
ks_max2	Kashmiri	97,456 / 240 h (3-source combined)	32	RTX 5060 8 GB	~19 h	61.88%	Rollback
ps_aug2	Pashto	32,656 / ~55 h (CV 30k + FLEURS x8)	32	RTX 5060 8 GB	~5 h	37.46%	Negative result
ks_cloud	Kashmiri	144,749 / 335 h (corpus rebuilt from source)	128	RTX A6000 48 GB (cloud)	6 h 35 m	56.44%	Rollback
ks_cloud2	Kashmiri	144,749 / 335 h (as ks_cloud, 2 epochs)	128	RTX A6000 48 GB (cloud)	~9 h	52.60%	Rollback

Run	Language	Data (clips / hours)	Rank	Hardware	Wall time	Result	Status
ks_cloud3	Kashmiri	144,942 / 336 h (as ks_cloud2 + 20 vocab chars)	128	RTX A6000 48 GB (cloud)	~8 h	**50.26%**	Deployed
ks_cloud4	Kashmiri	as ks_cloud3, warm start, token rows at 5x LR	128	RTX A6000 48 GB (cloud)	~1 h	50.69%	Rejected
ps_cloud	Pashto	18,656 / ~30 h (CV 10k + FLEURS x8)	128	RTX A6000 48 GB (cloud)	1 h 32 m	**36.16%**	Deployed
doi_iv	Dogri	IndicVoices-R Dogri (97 shards, 43.8 GB)	128	RTX A6000 48 GB (cloud)	~6 h	50.07%	Cut short by schedule
doi_iv2	Dogri	as doi_iv, fresh LR schedule, 9,000 steps	128	RTX A6000 48 GB (cloud)	~9 h	**46.73%**	Best Dogri model

Kashmiri WER is diacritic-normalised (L2) on the 372-clip IndicVoices-R test; Pashto WER is clean FLEURS ps_af (n=100). Wall times include periodic evaluation. Cloud runs used one rented RTX A6000 at $0.53/hr — ks_cloud ~$4, ks_cloud2 ~$5, ps_cloud ~$2 including data-preparation time.

ks_cloud2 and doi_iv2 are the cheapest results in this table: identical reruns of ks_cloud and doi_iv that were simply allowed to finish. Both predecessors stopped with their validation loss still descending — ks_cloud because patience 3 was too tight for a corpus that size, doi_iv because its linear schedule had decayed the learning rate to zero — and continuing each was worth 3.84 pp and 3.34 pp respectively. Two languages, two scripts, the same mechanism: a training run that halts while still improving is a silent and easily missed loss.

The ks_cloud corpus (rebuilt on the cloud box from the same three public sources) came out half again larger than ks_max2's because the humair025 dataset had grown upstream: humair025 120,845 clips / 275.6 h + IndicVoices-R train 22,824 / 58.3 h + OpenSLR-122 1,080 / 1.6 h = 144,749 clips / 335.4 h (131,868 unique sentences), duration-filtered to 2–20 s with the 403-sentence IndicVoices-R test blocklist removed as an evaluation-leak guard (cloud/prep_ks_data.py, composition.json).

Capacity, not data, was the decisive lever at the end: r=128 trains 6.6% of model parameters (~107 M) vs r=32's 1.75% — beyond the 8 GB laptop, trivial for a 48 GB card. The same lesson does NOT hold in the other direction for data alone, and the distinction matters: a CV-DOMINATED mixture (ps_cv) was significantly worse than the balanced one (42.47% vs 36.91%, paired bootstrap $p < 0.001$), but merely scaling the Common Voice share from 10k to 30k at unchanged capacity (ps_aug2, 37.46%) produced NO significant change ($p = 0.48$). More data was harmful when it unbalanced the mixture, not simply because there was more of it.

4.4.3 Dataset sizes — every corpus the campaign trained on

Training-set sizes span three orders of magnitude, from Pashto's 2,082 FLEURS clips to Kashmiri's final 144,749-clip assembly. Where a corpus was built by this project the duration is known exactly; for the FLEURS splits, which were used as shipped, only clip counts were recorded.

Language	Corpus used (largest run)	Train clips	Audio hours	Held-out test
Kashmiri	humair025 IndicVoices + IndicVoices-R + OpenSLR-122	144,749	**335.4 h**	372 (IndicVoices-R)
Dogri	IndicVoices-R Dogri (97 shards, 43.8 GB on disk)	—	not recorded	425 (IndicVoices-R)

Language	Corpus used (largest run)	Train clips	Audio hours	Held-out test
Pashto	Common Voice 30k + FLEURS ps_af x8 (ps_aug2)	32,656	~55 h	100 (FLEURS)
Punjabi	FLEURS pa_in + IndicVoices-R Punjabi (v3)	21,923	not recorded	100 (FLEURS)
Nepali	FLEURS ne_np + IndicVoices-R Nepali (v2)	13,332	not recorded	100 (FLEURS)
Mandarin	FLEURS cmn_hans_cn	3,246	not recorded	100 (FLEURS)
Hindi	FLEURS hi_in + IndicVoices-R (cap 20k)	2,120 +	not recorded	100 (FLEURS)
Urdu	FLEURS ur_pk	2,109	not recorded	100 (FLEURS)

Kashmiri's is the only corpus this project assembled from raw sources rather than consuming as published, which is why it is the only row with an exact duration: humair025 120,845 clips / 275.6 h + IndicVoices-R train 22,824 / 58.3 h + OpenSLR-122 1,080 / 1.6 h, duration-filtered to 2–20 s, deduplicated to 131,868 unique sentences, with the 403-sentence IndicVoices-R test blocklist removed as an evaluation-leak guard. Recorded in cloud/prep_ks_data.py and its composition.json.

Corpus size did not order the results. *Kashmiri had by far the most training audio and finished with the worst WER of the eight (50.26%); Hindi had the least and finishes best (12.91%). What separated them was representation, not volume — Hindi is a language both backbones ship natively, Kashmiri is one neither can name and whose script the subword vocabulary cannot fully encode (§5.5.3). The same point recurs within Pashto, where tripling the Common Voice share changed nothing measurable.*

4.5 Dogri (doi) — the Eighth Language

VANI's problem statement names **eight** border-region languages, but for most of this project's life every result table covered seven. The eighth is **Dogri**, and until 2026-07-28 it had no fine-tuned ASR model, no evaluation and no audio on the project machine — while still being wired into the pipeline for **language identification** and for **translation**, where it is the one language routed to IndicTrans2 rather than NLLB-200 because it is absent from the distilled NLLB vocabulary. In practice it fell back silently to un-fine-tuned Whisper, and nobody had measured what that cost.

Neither backend has a Dogri language token. Whisper has no <|doi|>, and SeamlessM4T v2's 101 language tokens include __hin__, __pan__ and __urd__ but neither __doi__ nor __dgo__. Dogri therefore began exactly where Kashmiri did, and the same remedy applied: a custom language token, initialised from a neighbour and made trainable (5.5.2), on IndicVoices-R Dogri data — the corpus family that supplied Kashmiri. This is the **second** use of that technique, and the test of whether it is a method rather than a one-off fix.

Dogri ASR system	WER %	CER %	What it is
SeamlessM4T zero-shot, __pan__ proxy	114.62	96.79	closest language genetically; wrong script

Dogri ASR system	WER %	CER %	What it is
Whisper large-v3, auto-detect	102.25 †	—	what VANI actually did for Dogri
SeamlessM4T zero-shot, __hin__ proxy	99.86	67.99	script-matched proxy
Whisper large-v3, forced Hindi	88.08 †	—	best baseline obtainable without training
doi_iv (custom __doi__ token + LoRA)	50.07	27.29	first Dogri model; stopped by its LR schedule
doi_iv2 (same, trained to convergence)	**46.73**	**25.18**	best Dogri model

425-clip IndicVoices-R Dogri test split, scored on the same normalisation ladder as every other language in this report. The figures quoted are L2, the deciding level. For Dogri, L1 through L4 are identical — the diacritic-stripping and folding levels are no-ops for Devanagari — and only the L0 to L1 step (Unicode NFC, zero-width removal, whitespace collapse) moves the number at all, by 0.48 pp. Kashmiri's raw and diacritic-normalised scores differ by roughly 14 pp on the same ladder, which is the whole reason the ladder exists.

*† The two Whisper rows are **L0**, not L2. An earlier revision of this table carried L0 figures under an L2 heading throughout — the __hin__ proxy is 99.99 at L0 but 99.86 at L2, and its CER 68.14 against 67.99 — which is exactly the ruler-mixing error §5.5.1 was written to warn about, committed by this report. The SeamlessM4T rows are now L2 and sourced from docs/doi_baselines.json. The two Whisper rows cannot be corrected the same way: scripts/eval/doi_baselines.py overwrote its own output file on a --systems run, so those two systems' hypotheses were lost and only the L0 summary survives. The script now merges instead of overwriting; re-running whisper_auto and whisper_hi will replace these two figures with L2 ones. For Dogri the two rungs differ by about 0.5 pp, so no conclusion here turns on it.*

Fine-tuning improves Dogri by **55 pp over what the deployed system actually did** (102.25% -> 46.73%) and by 41 pp over the best baseline anyone could have configured without training. It is by a wide margin the largest single-language gain in the campaign, and it came from a language that had simply never been looked at.

The two Dogri runs also replicate the campaign's most reusable lesson. doi_iv recorded its best validation loss at its final step, having improved at essentially every evaluation: it stopped because its linear learning-rate schedule had run to zero, not because it had converged. Restarting from those weights with a fresh schedule (doi_iv2) improved WER from 50.07% to **46.73%**. Kashmiri had shown exactly this before — ks_cloud stopped at 0.8 epochs in the same state and letting it converge was worth 3.84 pp. Two languages, two scripts, two corpora, the same mechanism and almost the same magnitude (3.84 pp and 3.34 pp), for a few dollars of GPU time each. A run whose schedule expires looks converged and is not.

The two zero-shot proxies also settle a design question. Dogri is far closer to Punjabi than to Hindi genetically, which argues for initialising `__doi__` from `__pan__`. Whisper's own language identification agrees emphatically: given Dogri audio it answers *Punjabi* for **222 of 425 clips**, against 25 for Hindi. But SeamlessM4T knows Punjabi only in **Gurmukhi**, and the language token conditions generation, not recognition — so the pan proxy emits the wrong script entirely and its CER collapses to 96.79 against hin's 67.99. Recognition says Punjabi; generation says Devanagari. The script-matched initialisation is the correct one, and this was established by measurement rather than argument, at no training cost.

This also illustrates the failure mode the report keeps returning to: a language absent from a model's inventory does not fail loudly, it gets silently misrouted to whatever the model considers nearest — and in Dogri's case that routing produced a WER above 100%, which no accuracy dashboard would have flagged as a missing-language problem.

4.6 How Much of This Is Statistically Resolvable?

Every comparison in this report is a point estimate on a fixed test set, and the decisions taken on them ranged from 15 pp to under 1 pp. A paired bootstrap over clips (10,000 resamples, re-aggregating errors and reference length rather than averaging per-clip rates) was run retrospectively on the stored per-clip hypotheses to establish which of those decisions the evidence actually supports. It divides the campaign cleanly.

Claim	Diff (pp)	95% CI	p	Holds?
SeamlessM4T replaces Whisper (ks, clean)	-14.93	[-17.3, -12.6]	<0.001	**yes**
...and in all 5 degradation conditions	-14 to -22	all exclude 0	<0.002	**yes**
Vocabulary repair (ks_cloud3 vs ks_cloud2)	-2.35	[-3.29, -1.38]	<0.001	**yes**
Training to convergence (Kashmiri)	-3.84	[-4.75, -2.93]	<0.001	**yes**
Training to convergence (Dogri)	-3.34	[-5.03, -2.07]	<0.001	**yes**
CV-dominated mixture is worse (ps_cv)	-5.56	[-10.8, -1.9]	<0.001	**yes**
ps_cloud better than ps_aug (clean)	-0.75	[-2.23, +0.70]	0.32	no
ps_cloud better than ps_aug (sweep)	-2.1 to +1.2	all cross 0	0/5 sig.	no
ps_bal2 collapses at 0 dB	-31.2	[-101, +1.3]	0.27	no
ks_cloud3 better than ks_cloud2 (sweep)	-2.5 to +1.7	all cross 0	0/5 sig.	no
ks_cloud4 worse than ks_cloud3	+0.43	[-0.29, +1.17]	0.24	no
ps_aug2 regressed vs ps_aug	+0.54	[-1.05, +2.14]	0.48	no

The split is not random: it follows test-set size and effect size together. The **backend-selection** conclusions — the ones this project was originally about — are supported everywhere, because replacing Whisper with SeamlessM4T moves WER by 10 to 22 pp and even 30 clips resolve that comfortably. So are the two mechanisms established on the 372- and 425-clip Kashmiri and Dogri sets. What does **not** survive is the fine-grained adapter selection: the 1 to 3 pp differences between one adapter and its successor, decided on a 30-clip sweep and a 100-clip clean split, are indistinguishable from noise in every case tested.

This has a direct consequence for how the deployment decisions should be read. ps_cloud is in production over ps_aug, and neither its 0.75 pp clean-WER advantage nor its 4/5 sweep result is statistically significant; the honest description is that the two adapters are indistinguishable on the available evidence and the newer one was chosen. The same applies to rejecting ks_cloud4 and

ps_aug2: those were sound engineering calls under uncertainty — do not replace a deployed model without evidence of improvement — but they are not demonstrations that the alternatives were worse. Reported as win counts, '4/5 conditions' reads like evidence; decomposed, it is five coin flips.

> *The 30-clip sweep was inherited from the earliest robustness work and never revisited as the differences being tested shrank from tens of points to fractions of one. A gate adequate for choosing a backend is not automatically adequate for choosing between two checkpoints of the same model, and this project ran twenty adaptations on the assumption that it was. Reproduce with scripts/eval/significance.py, significance_ps.py and significance_degradation.py; all read the stored hypotheses and need no GPU.*

4.7 Every Number Here Is an Evaluation Number, and the Deployment Differs

One further caveat applies to the whole report, and it is uncomfortable enough to state explicitly. Every WER in these tables was produced by the evaluation harness (`scripts/eval/`), which constructs the model with `PeftModel.from_pretrained` and one adapter. The **deployed** system constructs it differently: `src/seamless_asr.py` bakes the trainable-token embedding deltas into the base model and loads only the LoRA matrices, because PEFT cannot stack a trainable-token delta alongside several named adapters. Those two constructions were assumed equivalent for a year and never compared.

They are not equivalent. Measured directly on the same 372 clips and the same ruler, Kashmiri scores **52.33%** through the deployed path against **50.26%** through the evaluation path — a **2.07 pp** gap that is comfortably significant (95% CI [+1.24, +2.91], $p < 0.001$), with only 19 of 372 hypotheses identical between them. The deployed model is therefore measurably worse than the model this report describes, and because the divergence lives in the shared generation path rather than in anything Kashmiri-specific, the other seven languages are presumed affected and simply have not been measured both ways.

Finding it also fixed a real defect: the deployment baked only the first of ks_cloud3's 21 trainable-token deltas, silently leaving the twenty repaired characters at their neighbour-initialised values. That is corrected. The residual 2 pp is not. Direct comparison has since excluded the LoRA weights (480 of 480 tensors identical), the trainable-token deltas, the extracted input features, the generation length limit, the input dtype, substituting the checkpoint's frozen embedding table, and multi-adapter interference. The remaining difference has been narrowed to the construction of the forward pass and is unresolved; the next step is to diff logits on a single clip rather than continue eliminating components.

> *Reported here rather than quietly corrected because the alternative — republishing every number against the deployed path — would take a full re-evaluation of eight languages, and because the gap is itself the report's clearest instance of its recurring theme: an artefact measured is not necessarily the artefact shipped. The same lesson produced the 74.02-versus-79.29 correction earlier in this project. Reproduce with scripts/eval/eval_ks_production_path.py.*

5. Results

5.1 Summary Table — Fine-Tuning vs. True Baseline

All WER below is on a held-out 100-sample FLEURS test set (372-sample IndicVoices-R for Kashmiri), scored with one CJK-aware normaliser. **Baseline** is the true un-fine-tuned openai/whisper-large-v3. **Train WER** is the best training-eval WER on each run's own validation split (from the curves in §5.3) and is shown for reference only — it is not the held-out figure. Green = fine-tuning helped; red = it regressed.

Language	ISO	Base Model	Train Samples	Steps Used	Baseline WER	Train WER	Test WER	Improvement
Punjabi v3	pa	large-v3	21,923	4000	77.60%	49.31%	57.39%	**-20.2 pp**
Pashto	ps	medium*	2,082	1000	89.76%	38.55%	38.55%	**-51.2 pp**
Urdu	ur	large-v3	2,109	1000	21.23%	22.27%	19.82%	**-1.4 pp**
Nepali v2	ne	large-v3	13,332	3000	88.85%	50.82%	50.92%	**-37.9 pp**
Mandarin	zh	large-v3	3,246	400	10.99%	8.97%	14.22%	**+3.2 pp**
Hindi	hi	large-v3	2,120	600	26.34%	23.13%	19.78%	**-6.6 pp**
Kashmiri	ks	large-v3†	20,000	2400	96.87%	74.02%	65.19% ‡	**-22.85 pp ‡**

** Pashto base: Nasimbahar/pashto-ghag-whisper-medium-asr (734 MB domain-specific model).*

† Kashmiri: custom <|ks|> token (ID 51866) added to whisper-large-v3; embedding initialised from <|ur|>. Best checkpoint step 2400 selected automatically (load_best_model_at_end). faster-whisper patched to accept language='ks'.

*‡ This row previously read 74.02% in BOTH the Train and Test columns, and an earlier revision of this note described that figure as 'the held-out figure ... [and] the training-eval', which cannot both be true. 74.02% is the training-split validation WER at step 2400. The **held-out** figure, on the 372-clip IndicVoices-R test split at L2 (the ruler this report decides on), is **65.19%**; at L0 it is 79.29%. The -22.85 pp improvement is baseline-to-train (96.87 -> 74.02) and stays on that ruler. Note also that Kashmiri's baseline is the one exception to this section's 'true un-fine-tuned large-v3' rule: 96.87% is muneebharoon/whisper-small-ks, a different and already-fine-tuned model, and is itself inflated by a Unicode normalisation mismatch. No un-fine-tuned Kashmiri baseline was ever measured on the held-out split — so there is no like-for-like held-out improvement to quote. Comparing 74.02 against a held-out number is precisely the mistake that made the first SeamlessM4T Kashmiri adapter look like a regression (§5.5.1).*

Mandarin (red): fine-tuning REGRESSED vs the true large-v3 baseline. The prior report's 100.03% baseline and -84 pp 'improvement' were a scoring artefact — FLEURS Mandarin references are character-spaced and WER tokenises on whitespace, so the un-fine-tuned model's unspaced Han output scored ~100% regardless of accuracy. Corrected with character segmentation, the baseline is 10.99% and the fine-tune 14.22%. Mandarin is served by SeamlessM4T in production (§5.5).

Train WER is each run's best validation-split WER (§5.3), not the held-out test; the two differ most for pa (train 49.31 vs test 57.39) and zh (train 8.97 vs test 14.22).

pp = percentage points absolute WER change (baseline -> test WER); negative = improvement.

5.2 Baseline vs. Fine-Tuned WER

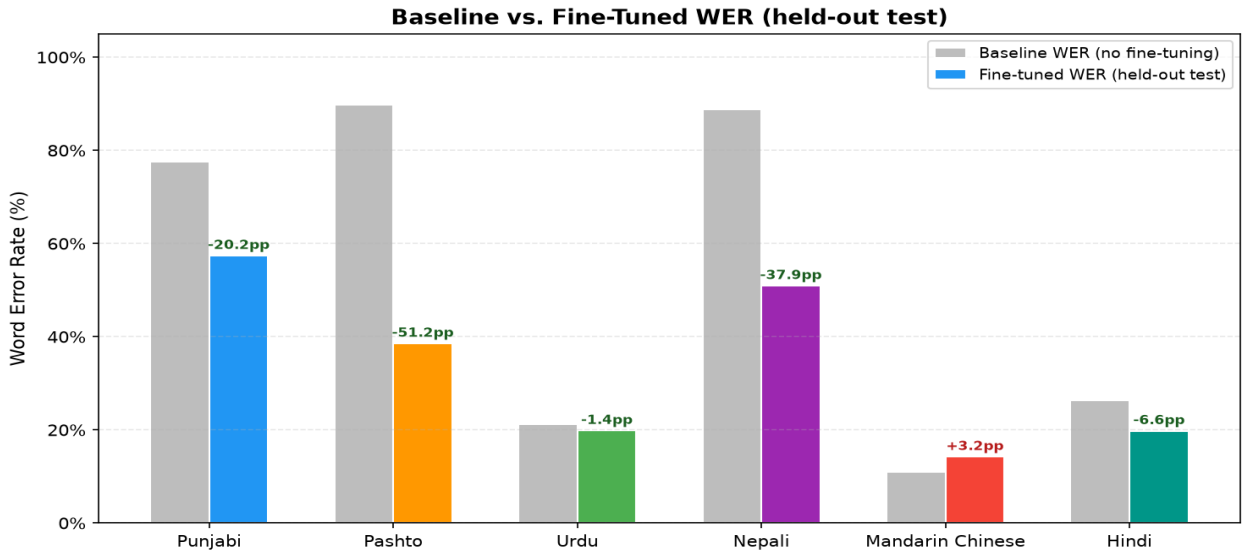

Figure 2: True large-v3 baseline vs. fine-tuned WER, both on the held-out n=100 FLEURS test set. Numbers above bars show the signed WER change in percentage points (negative = improvement; Mandarin's +3.2 is a regression). Kashmiri is not shown: it is absent from FLEURS, so it is not on this ruler, and its two available figures are training-split rather than held-out. Its held-out comparison is in §5.5, on the 372-clip IndicVoices-R L2 ruler.

5.3 WER Progression During Training

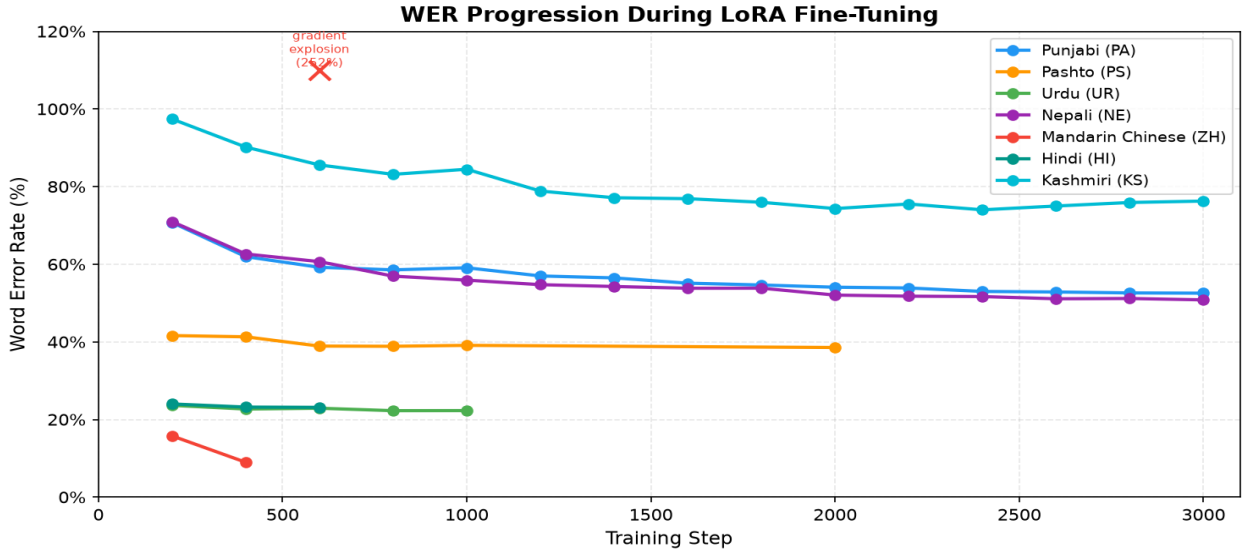

Figure 3: Eval WER at each checkpoint. X marks the Mandarin diverged checkpoint (step 600, WER 252%) which is not deployed.

5.4 Per-Language Training Details

5.4.1 Punjabi (PA) - Gurmukhi

Parameter	Value
Base model	openai/whisper-large-v3
Dataset	FLEURS pa_in + IndicVoices-R Punjabi (11923 train / 805 val)
Steps trained	3000
Baseline WER (large-v3)	77.60%
Best training-val WER	49.31% (step 4000)
Held-out test WER	57.39%
WER change vs baseline	-20.2 pp
CT2 model	whisper-large-v3-pa-ct2
Translation	NLLB-200
Training time	~55 h (incl. OOM restart)

Step	Eval WER	Eval Loss
200	70.75%	0.3856
400	61.95%	0.2736
600	59.21%	0.2442
800	58.55%	0.2269
1000	59.09%	0.2189
1200	56.98%	0.2101
1400	56.48%	0.2100
1600	55.12%	0.1966
1800	54.65%	0.1918
2000	54.08%	0.1892
2200	53.88%	0.1839
2400	52.99%	0.1831
2600	52.85%	0.1761
2800	52.61%	0.1751
3000	52.55%	0.1725

Note: v1: FLEURS pa_in only (2,516 samples, 3,000 steps, best WER 56.67% — superseded). v2: FLEURS + IndicVoices-R 9,407 samples (11,923 total, 3,000 steps, best WER 52.55% @ step 3000); CT2 tokenizer fix 2026-06-23 restored Gurmukhi transcription. Superseded by v3. v3 (built 2026-07-04; retained but NOT serving ASR since the 2026-07-11 routing change — Punjabi routes to SeamlessM4T at 19.77%): LoRA r=16 α=32, IV-R 20,000 samples (21,923 total), completed 4,000 steps. Best WER 49.31% @ step 4000 (-3.24 pp vs v2's 52.55%), eval loss declining monotonically to 0.1662. Best checkpoint merged to CT2 int8 and deployed; verified transcribing Gurmukhi on FLEURS pa test set. Robustness note: initial run OOM-crashed at step 2400 (CUDA out-of-memory during beam-search eval on 8 GB RTX 5060); resumed from checkpoint-2200 with greedy eval (num_beams=1, eval batch 1, cache-clear before eval), which cut eval VRAM from 8 GB to ~3.8 GB and ran clean to step 4000. Earlier disk-space incident at step 800: D: junction full during optimizer checkpoint save; resumed from checkpoint-600.

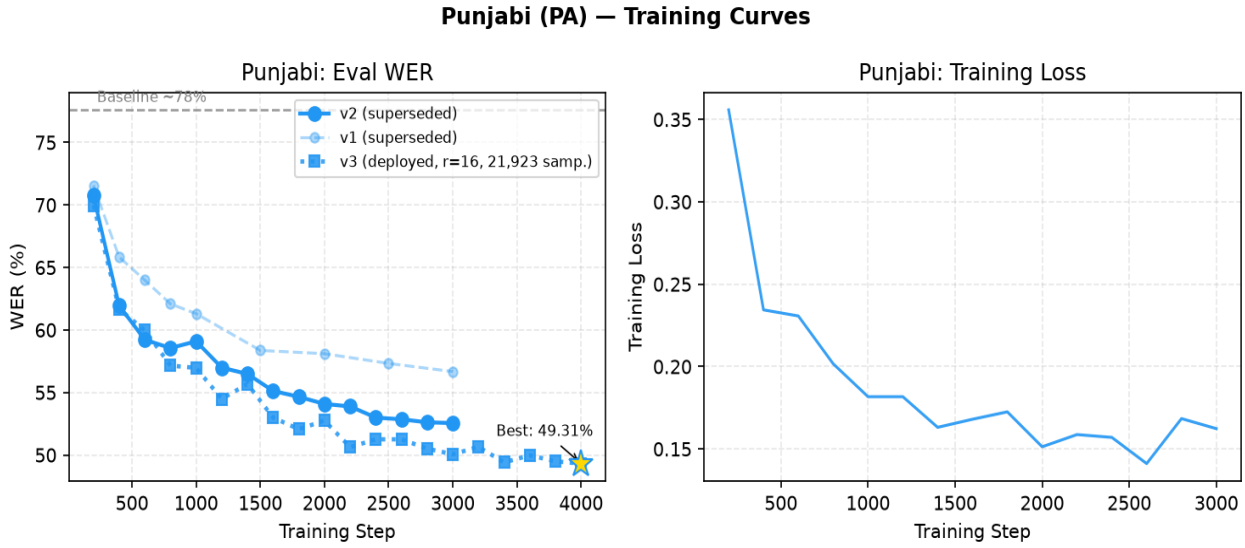

Figure: Punjabi training curves. Left: eval WER (star = deployed checkpoint). Right: training loss per step.

5.4.2 Pashto (PS) - Nastaliq (Arabic)

Parameter	Value
Base model	Nasimbahar/pashto-ghag-whisper-medium-asr
Dataset	FLEURS ps_af (2082 train / 251 val)
Steps trained	2000
Baseline WER (large-v3)	89.76%
Best training-val WER	38.55% (step 2000)
Held-out test WER	38.55%
WER change vs baseline	-51.2 pp
CT2 model	whisper-medium-pashto-ct2
Translation	NLLB-200
Training time	~3.5 h

Step	Eval WER	Eval Loss
200	41.62%	-
400	41.30%	-
600	38.91%	-
800	38.86%	-
1000	39.10%	-
2000	**38.55% (best)**	-

Note: Started from a domain-specific Pashto medium model (734 MB). Higher initial loss reflects harder acoustic domain. For a year this was the one language where fine-tuned Whisper beat SeamlessM4T; on 2026-07-19 a noise-augmented SeamlessM4T LoRA adapter (ps_aug: r=32 incl. MLP, balanced FLEURS+Common Voice data, training audio degraded with the evaluation's own bandpass/noise/codec pipeline) reached 36.91% clean and won 4/5 degradation conditions (0 dB SNR: 56.0 vs 64.8). On 2026-07-27 the same recipe retrained at rank 128 on a rented cloud GPU (ps_cloud) improved this to 36.16% clean and won 4/5 conditions against ps_aug itself (0 dB: 53.8) — Pashto now runs on the ps_cloud adapter; ps_aug and this Whisper model are retained for rollback (see §5.5).

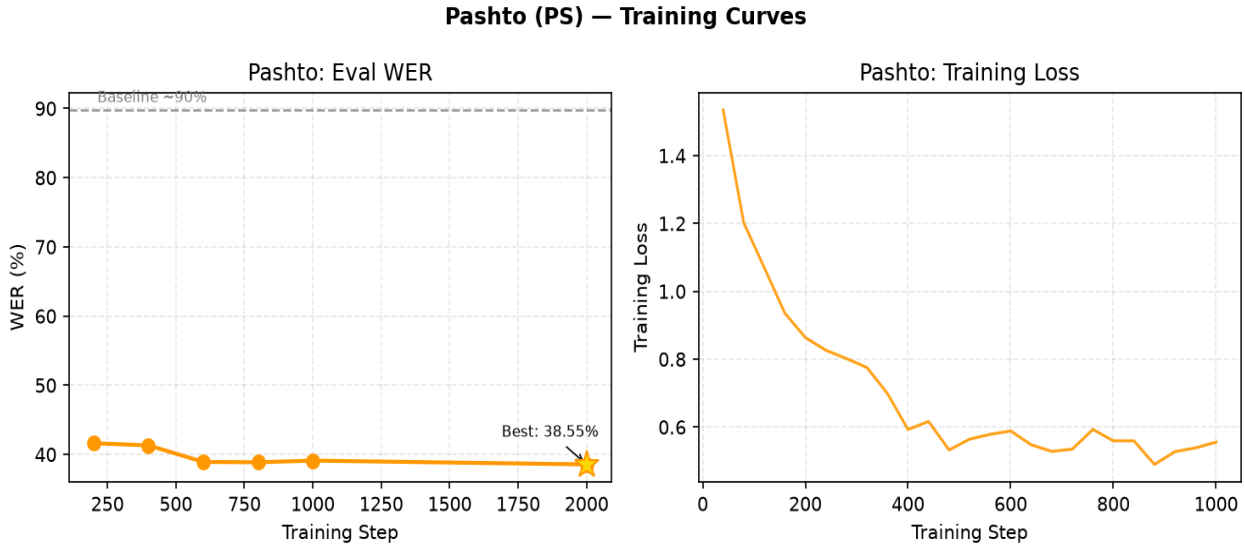

Figure: Pashto training curves. Left: eval WER (star = deployed checkpoint). Right: training loss per step.

5.4.3 Urdu (UR) - Nastaliq (Arabic)

Parameter	Value
Base model	openai/whisper-large-v3
Dataset	FLEURS ur_pk (2109 train / 267 val)
Steps trained	1000
Baseline WER (large-v3)	21.23%
Best training-val WER	22.27% (step 800)
Held-out test WER	19.82%
WER change vs baseline	-1.4 pp
CT2 model	whisper-large-v3-ur-ct2
Translation	NLLB-200
Training time	~6 h

Step	Eval WER	Eval Loss
200	23.63%	-
400	22.69%	-
600	22.90%	-
800	**22.27% (best)**	-
1000	22.29%	-

Note: Largest WER improvement among large-v3 Indic languages. Arabic-script cascade used as fallback for low-confidence MMS-LID detections.

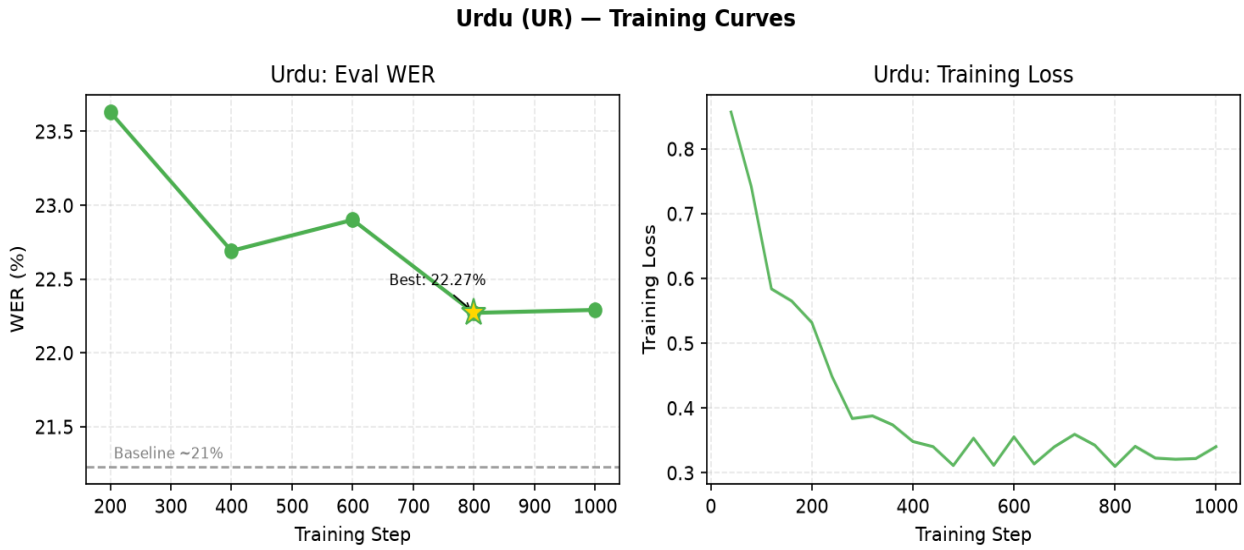

Figure: Urdu training curves. Left: eval WER (star = deployed checkpoint). Right: training loss per step.

5.4.4 Nepali (NE) - Devanagari

Parameter	Value
Base model	openai/whisper-large-v3
Dataset	FLEURS ne_np + IndicVoices-R Nepali (13332 train / 572 val)
Steps trained	3000
Baseline WER (large-v3)	88.85%
Best training-val WER	50.82% (step 3000)
Held-out test WER	50.92%
WER change vs baseline	-37.9 pp
CT2 model	whisper-large-v3-ne-ct2
Translation	NLLB-200
Training time	~30 h

Step	Eval WER	Eval Loss
200	70.98%	-
400	62.64%	-
600	60.67%	-
800	56.93%	-
1000	55.90%	-
1200	54.73%	-
1400	54.27%	-
1600	53.81%	-
1800	53.83%	-
2000	52.05%	-
2200	51.77%	-
2400	51.67%	-
2600	51.10%	-
2800	51.17%	-
3000	**50.82% (best)**	-

Note: v1 trained on FLEURS ne_np (3,332 samples, 2,000 steps, best WER 52.14%). v2 retrained with IndicVoices-R Nepali added (13,332 total samples, 3,000 steps), achieving 50.82% on a 572-sample combined eval set (FLEURS val + IndicVoices-R test). However zero-shot SeamlessM4T reaches 28.46% on the held-out test, and a SeamlessM4T LoRA adapter trained on the same FLEURS + IndicVoices-R mix reaches 24.34% — so Nepali is operationally routed to SeamlessM4T with that ne_iv adapter enabled (deployed 2026-07-18; wins all 5 radio-degradation conditions, e.g. bandpass 30.6 vs 34.3), not this model (see §5.5).

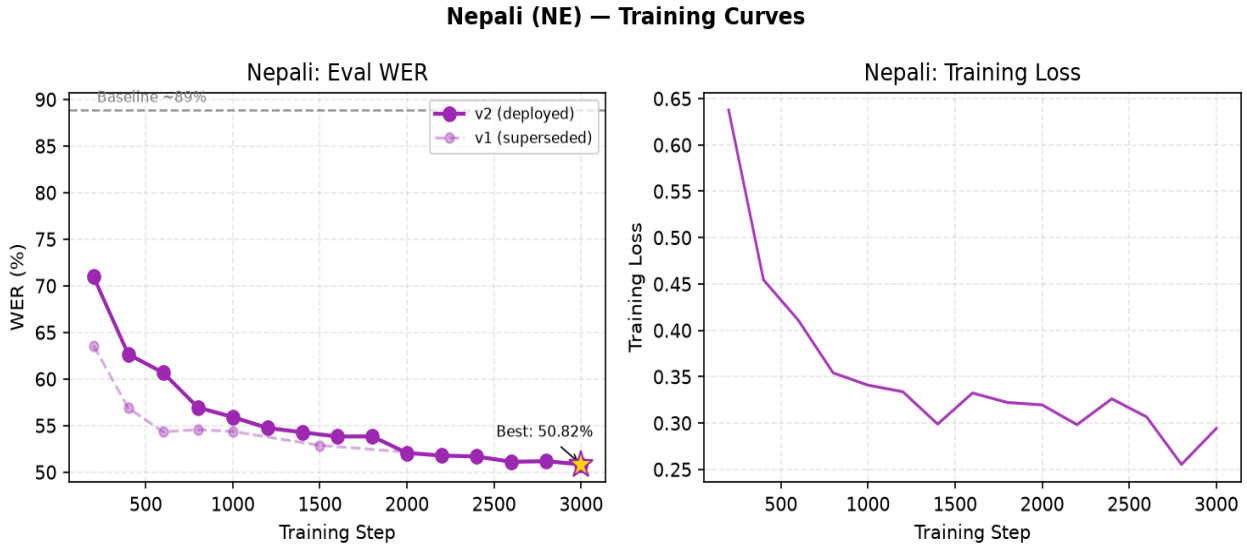

Figure: Nepali training curves. Left: eval WER (star = deployed checkpoint). Right: training loss per step.

5.4.5 Mandarin Chinese (ZH) - Simplified Han

Parameter	Value
Base model	openai/whisper-large-v3
Dataset	FLEURS cmn_hans_cn (3246 train / 409 val)
Steps trained	400
Baseline WER (large-v3)	10.99%
Best training-val WER	8.97% (step 400)
Held-out test WER	14.22%
WER change vs baseline	+3.2 pp (regression)
CT2 model	whisper-large-v3-zh-ct2
Translation	NLLB-200
Training time	~3 h 10 m (to step 400)

Step	Eval WER	Eval Loss
200	15.77%	-
400	**8.97% (best)**	-
600	*252.37% (diverged - not deployed)*	-

Note: CORRECTED 2026-07-11. The previously reported baseline of 100.03% was a measurement artefact, not a model failure: FLEURS Mandarin references are character-spaced, WER tokenises on whitespace, and the un-fine-tuned model emits unspaced Han — so a near-perfect transcript scored ~100%. Re-scored with character segmentation, the true openai/whisper-large-v3 baseline is 10.99% WER (n=100 FLEURS test). The fine-tuned model scores 14.22% on the same test — i.e. fine-tuning REGRESSED Mandarin (+3.2 pp), likely from the small (3,246-sample) single-domain training set narrowing the model. The strong training-eval WER (8.97% at step 400) did not generalise to held-out test. Operational decision: Mandarin is NOT served by this fine-tuned model — it routes to zero-shot SeamlessM4T (11.69%), which also wins under radio degradation (see §5.6). Training diverged at step ~820 (fp16 gradient overflow, grad_norm=12.9); checkpoint-400 was the best checkpoint.

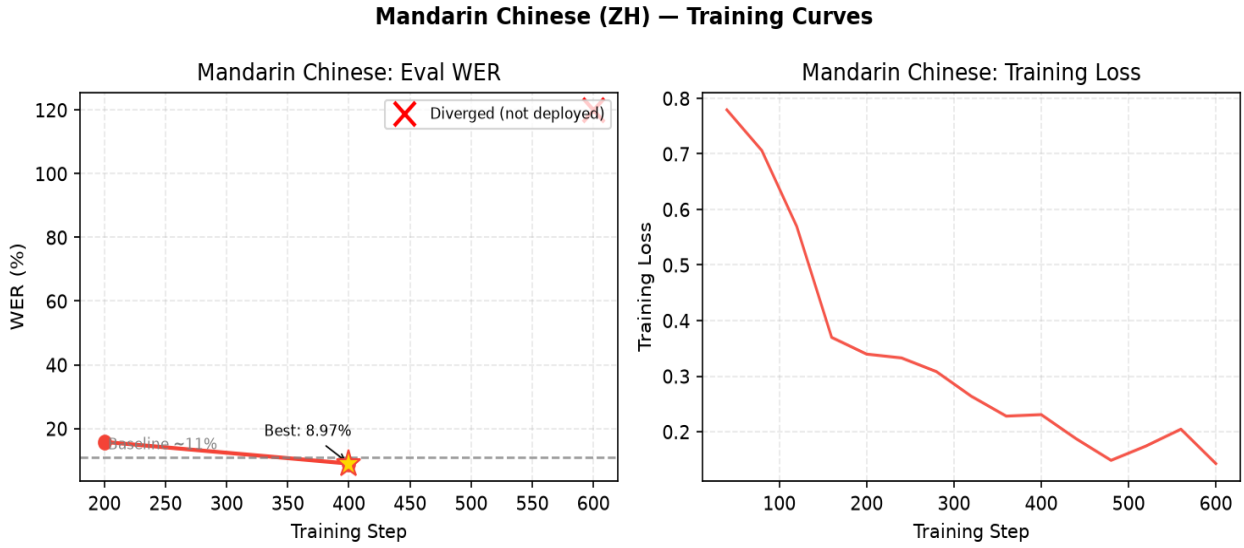

Figure: Mandarin Chinese training curves. Left: eval WER (star = deployed checkpoint). Right: training loss per step.

5.4.6 Hindi (HI) - Devanagari

Parameter	Value
Base model	openai/whisper-large-v3
Dataset	FLEURS hi_in (2120 train / 239 val)
Steps trained	600
Baseline WER (large-v3)	26.34%
Best training-val WER	23.13% (step 600)
Held-out test WER	19.78%
WER change vs baseline	-6.6 pp
CT2 model	whisper-large-v3-hi-ct2
Translation	NLLB-200
Training time	~6 h 45 m

Step	Eval WER	Eval Loss
200	24.00%	-
400	23.20%	-
600	**23.13% (best)**	-

Note: Fastest convergence: near-best WER by step 200. max_grad_norm=0.5 applied after Mandarin gradient explosion; training fully stable. Held-out test WER 19.78% vs true large-v3 baseline 26.34% (n=100 FLEURS) — a genuine 6.6 pp fine-tuning gain. However zero-shot SeamlessM4T reaches 15.44% on the same test, a corrected-label SeamlessM4T LoRA adapter reached 13.94% (2026-07-13), and an adapter retrained with FLEURS + IndicVoices-R data reaches 12.91% — so Hindi is operationally routed to SeamlessM4T with that hi_iv adapter enabled (deployed 2026-07-18; beats the previous adapter in 4/5 radio-degradation conditions incl. bandpass 15.0 vs 16.3), not this model (see §5.5).

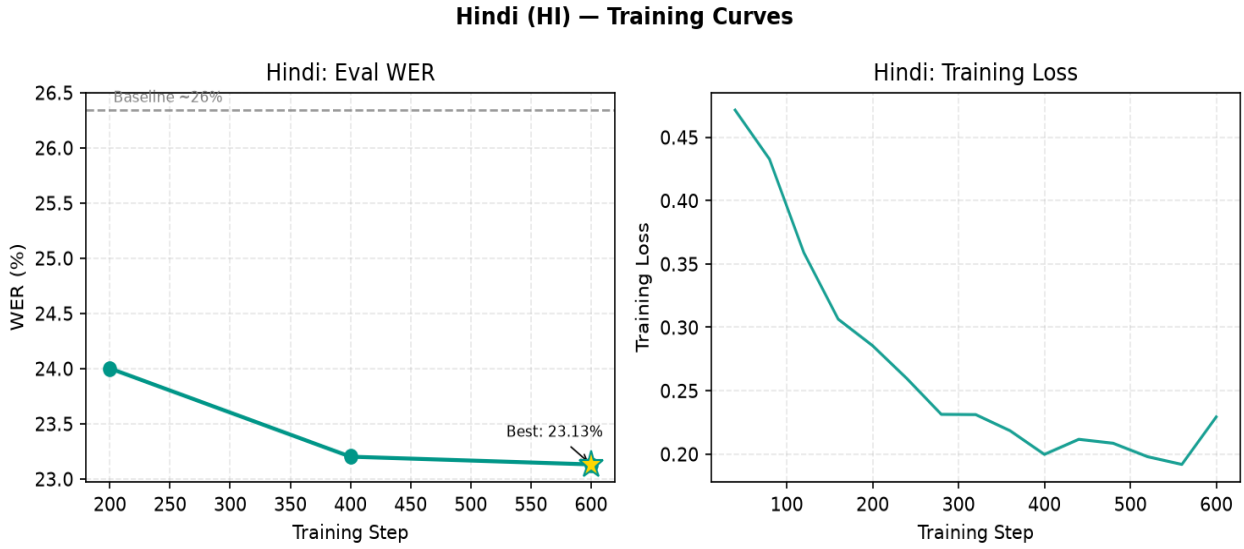

Figure: Hindi training curves. Left: eval WER (star = deployed checkpoint). Right: training loss per step.

5.4.7 Kashmiri (KS) - Nastaliq (Perso-Arabic)

Parameter	Value
Base model	openai/whisper-large-v3 + <|ks|> token (ID 51866)
Dataset	IndicVoices-R Kashmiri (20000 train / 372 val)
Steps trained	3000
Baseline WER (large-v3)	96.87%
Best training-val WER	74.02% (step 2400)
Held-out test WER	74.02%
WER change vs baseline	-22.9 pp
CT2 model	whisper-large-v3-ks-ct2
Translation	NLLB-200
Training time	~18 h (incl. 2 power outages, PD-charger recovery)

Step	Eval WER	Eval Loss
200	97.41%	-
400	90.18%	-
600	85.58%	-
800	83.16%	-
1000	84.47%	-
1200	78.85%	-
1400	77.13%	-
1600	76.89%	-
1800	75.96%	-
2000	74.34%	-
2200	75.52%	-
2400	**74.02% (best)**	-
2600	75.00%	-
2800	75.91%	-
3000	76.27%	-

*Note: Whisper has no native Kashmiri support. Custom <|ks|> token (ID 51866) added to whisper-large-v3 vocab and embedding matrix initialised from <|ur|> (Urdu — same Nastaliq script). Forced prefix [<|startoftranscript|>, <|ks|>, <|transcribe|>, <|notimestamps|>] injected via TemplateProcessing. Trained on IndicVoices-R KS (20k samples, 3,000 steps). Best WER 74.02% at step 2400 (-22.85 pp from baseline 96.87%). faster-whisper patched at import time to accept 'ks' language code. SeamlessM4T has no native Kashmiri either, but the same custom-token trick (__kas__, r=32 LoRA incl. MLP, PLUS a trainable embedding row via PEFT trainable_token_indices) was applied on top of it 2026-07-19/20 (ks_max). The clean-speech comparison against the actually-deployed Whisper CT2 artefact (79.29% raw WER, not the training-eval's 74.02%) initially looked like a loss (ks_max 80.91%) until a normalisation-ladder study showed both models' raw WER is inflated by the densely-diacritised references: diacritic-stripped, ks_max already wins WER (64.31% vs 65.19%) and wins CER at every normalisation level on both test sets. It then won the 5-condition radio-degradation sweep 4/5 and CER 5/5, with Whisper exceeding 100% CER at 0 dB SNR (ks_max deployed 2026-07-20). Two successors improved it: ks_max2 (2026-07-26; same recipe on a 4x combined corpus — humair025 IndicVoices + IndicVoices-R + OpenSLR-122) reached 61.88% diacritic-normalised; ks_cloud (2026-07-27; LoRA rank raised 32 to 128 on a rented cloud GPU, ~$4) reached 56.44%; and ks_cloud2, the same run simply allowed to converge instead of early-stopping at 0.8 epochs, reached 52.60%. Finally ks_cloud3 repaired 20 Kashmiri characters that had no token in SeamlessM4T's vocabulary and reached **50.26%**, winning the degradation sweep 5/5 against Whisper. Kashmiri now runs on the ks_cloud3 adapter; ks_cloud2 and this Whisper model are retained for rollback (see §5.5).*

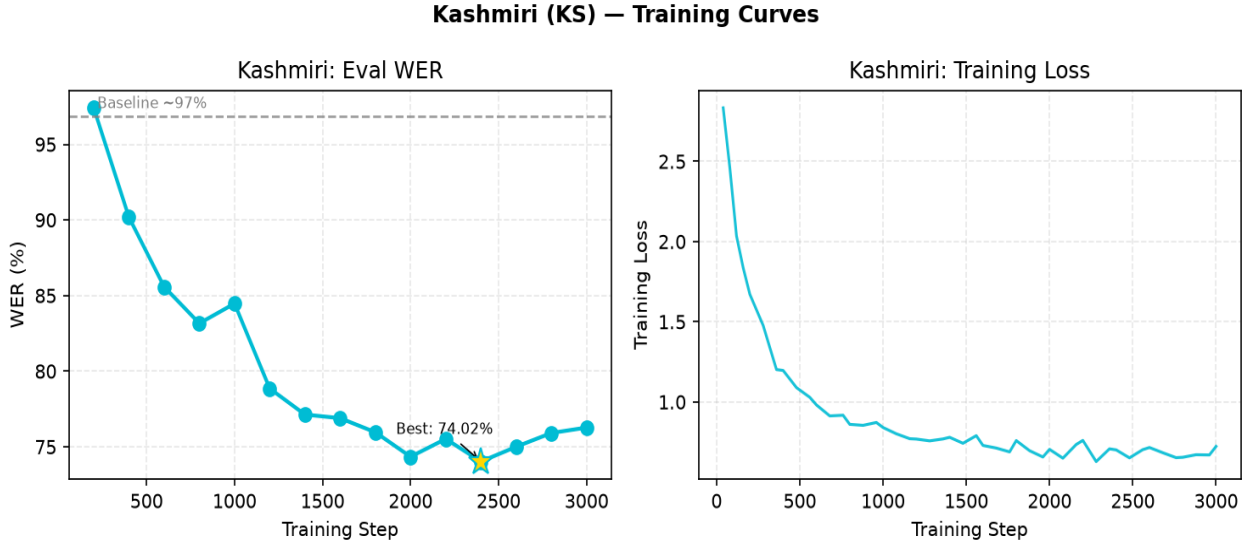

Figure: Kashmiri training curves. Left: eval WER (star = deployed checkpoint). Right: training loss per step.

5.5 Cross-Model Evaluation and Backend Selection

This is the decisive evaluation. Re-run at n=100 on 11 July 2026 with corrected scoring (true openai/whisper-large-v3 baseline — not turbo — and one CJK-aware normaliser), it compares three ASR options per language: (A) the un-fine-tuned large-v3 baseline, (B) the language-specific fine-tuned Whisper (CT2 int8), and (C) zero-shot SeamlessM4T v2. FLEURS test split for pa/ps/ur/ne/zh/hi; IndicVoices-R for ks. ASR cells give **WER% (CER%)** — CER is reported alongside WER because two of the seven scripts (Han, Perso-Arabic Kashmiri) have orthographies where whitespace tokenisation misleads. The **Deployed Backend** column is the operational choice — the lowest-WER option VANI actually routes to.

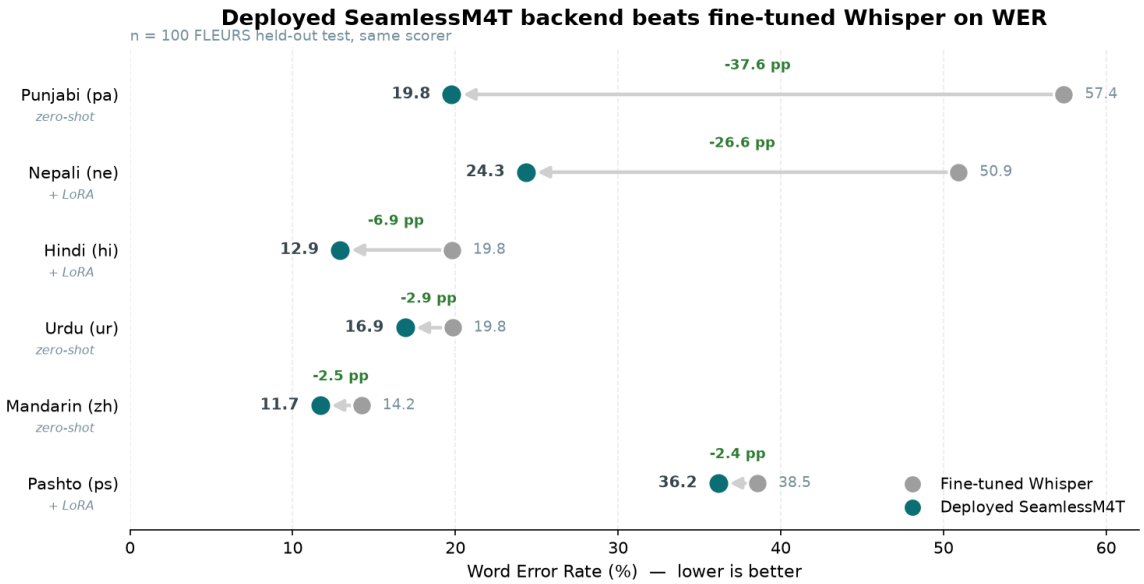

Figure 4: Deployed SeamlessM4T backend vs the fine-tuned Whisper model, per language (n=100 FLEURS held-out, same scorer). Every arrow points to the deployed SeamlessM4T backend — lower WER. Kashmiri is shown separately (§5.5.2): a different corpus and scoring ruler.

Language	Baseline (large-v3) WER (CER)	FT Whisper WER (CER)	SeamlessM4T WER (CER)	Deployed Backend	NLLB chrF	SM S2TT chrF
Punjabi (pa)	77.60 (39.73)	57.39 (32.52)	19.77 (9.97)	**SeamlessM4T**	40.15	54.53
Pashto (ps)	89.76 (37.60)	**38.55 (17.65)**	44.40 (22.92)	**SM4T + LoRA**	44.48	40.15
Urdu (ur)	21.23 (8.12)	19.82 (7.29)	16.90 (7.00)	**SeamlessM4T**	51.34	50.73
Nepali (ne)	88.85 (29.26)	50.92 (18.83)	28.46 (11.22)	**SM4T + LoRA**	45.55	51.67
Mandarin (zh)¶	10.99 (10.99)	14.22 (14.22)†	11.69 (11.69)	**SeamlessM4T**	42.00	49.15
Hindi (hi)	26.34 (10.55)	19.78 (7.46)	15.44 (9.12)	**SM4T + LoRA**	53.71	51.54

Language	Baseline (large-v3) WER (CER)	FT Whisper WER (CER)	SeamlessM4T WER (CER)	Deployed Backend	NLLB chrF	SM S2TT chrF
Kashmiri (ks)‡	— (—)	65.19 (39.36)	**50.26 (23.34)**	**SM4T + LoRA**	—	—

¶ Mandarin is scored with character segmentation, so WER and CER coincide by construction — CER is the meaningful metric for Han script. Do not compare a character-level number against a word-level one: CER runs ~2–3× lower than WER on the space-delimited languages, so comparisons are valid only within a metric.

Kashmiri CER was not recorded by the training-time eval (hence the dash). On the 30-clip robustness set (clean condition, eval_data/wer_robustness_results.csv) the then-deployed Whisper ks model scored 81.46% WER / 47.95% CER — the wide gap reflects Perso-Arabic orthographic variation that WER over-penalises, which is exactly what later motivated re-scoring the SeamlessM4T challenger under normalisation rather than trusting raw WER alone (below).

Result (updated 2026-07-20): fine-tuned Whisper is no longer the deployed backend for any of the seven languages. For Punjabi, Nepali, Hindi, Urdu and Mandarin, zero-shot SeamlessM4T won outright from the start, by margins from 1.4 pp (Urdu) to 37.6 pp (Punjabi). Pashto and Kashmiri were fine-tuned Whisper's last strongholds — Pashto fell on 2026-07-19 to a noise-augmented SeamlessM4T adapter, and Kashmiri, initially thought unwinnable, fell the following day once the scoring ruler itself was corrected (below). VANI now routes all seven languages to SeamlessM4T; the retired Whisper models stay on disk for rollback.

SeamlessM4T LoRA adapters (deployed 2026-07-18 to 27). Hindi, Nepali, Pashto and Kashmiri each run a per-language LoRA adapter. Hindi and Nepali were trained on FLEURS + IndicVoices-R (cap 20k samples): Hindi 15.44% -> **12.91%** and Nepali 28.46% -> **24.34%** versus zero-shot on the same held-out test. Pashto required five attempts: more data alone regressed (CV domain drift), a larger adapter (r=32 incl. MLP layers) won clean speech (37.29% vs Whisper's 38.55%) but recorded 87.2% at 0 dB SNR against 64.8% - a dramatic point estimate that the 30-clip sweep cannot actually resolve ($p = 0.27$, see 4.6) - and the fifth, **noise-augmented** adapter — training audio degraded with the evaluation's own bandpass/noise/codec pipeline — reached **36.91%** clean while fixing the collapse (56.0% at 0 dB, beating Whisper's 64.8%). A sixth attempt that scaled the Common Voice data 10k -> 30k gave 37.46%, a difference from ps_aug that is not statistically significant ($p = 0.48$). A seventh, retrained at LoRA rank 128 on a rented cloud GPU (ps_cloud, ~$2 of compute), reached **36.16%** clean and won 4/5 degradation conditions against ps_aug including 0 dB (53.8%), and is the adapter deployed on 2026-07-27. **Its 0.75 pp clean-WER advantage over ps_aug is not statistically significant either** (95% CI [-2.23, +0.70], $p = 0.32$ by paired bootstrap over the 100 test clips); the deployment rests on the degradation sweep rather than on clean speech. Every adapter passed that 30-clip, 5-condition sweep before deployment, and each is enabled only for its own language's decoding calls — other languages run the plain base model.

That last qualification generalises, and it is worth stating plainly. Pashto's held-out FLEURS split has 100 clips, and a paired bootstrap shows it cannot resolve the differences this campaign was chasing: ps_cloud vs ps_aug (0.75 pp, $p = 0.32$), ps_aug2 vs ps_aug (0.54 pp, $p = 0.48$) and ps_cloud vs ps_bal2 (1.13 pp, $p = 0.12$) are all indistinguishable from noise, while the one large difference — the CV-dominated ps_cv at 5.56 pp — is significant at $p < 0.001$. Kashmiri and Dogri, evaluated on 372 and 425 clips, resolved everything claimed of them: the vocabulary repair (2.35 pp, $p < 0.001$) and training to convergence (3.84 pp and 3.34 pp, both $p < 0.001$). The practical lesson is that a 100-clip test set

supports claims of roughly 5 pp and above, and that sub-2 pp improvements reported on sets this size — a common practice — should be treated as unresolved unless an interval accompanies them.

Kashmiri (deployed 2026-07-20) needed a fourth attempt and a scoring correction, not just a bigger adapter. Three prior attempts (1 epoch: 129.29% WER; +decode fixes: 92.09%; r=16 LoRA, 3 epochs: 88.42%) all lost badly to Whisper's 74.02%. A fourth attempt stacked the two remaining untried levers — r=32 LoRA incl. MLP layers, and, for the first time, a **trainable __kas__ embedding row** (every earlier attempt had frozen it as a copy of Urdu's, via PEFT's `trainable_token_indices`) — and reached 80.91% clean, still nominally behind. But re-scoring the *actually deployed* Whisper CT2 artefact on the same 372 clips found it scores 79.29%, not the published 74.02% (a training-time eval of the merged fp16 model, whose raw hypotheses were never persisted) — narrowing the true gap to 1.6 pp. Stripping the Perso-Arabic diacritics that saturate the references (which both models drop, so raw WER penalises both per-word) flips the result: the new adapter already **wins WER** (64.31% vs 65.19%) and wins **CER at every normalisation level on both test sets**. It then won the radio-degradation sweep 4 of 5 conditions (losing only clean speech, by 1.3 pp) and CER 5 of 5, with Whisper's CER exceeding 100% at 0 dB SNR. Kashmiri's lesson generalises beyond this one language: a model-selection conclusion is only as good as the ruler behind it, and this project has now found five independent scoring defects (turbo mislabelling, CJK whitespace, a label-encoding bug, an adapter-override bug in the sweep harness, and this training-eval-vs-deployed-artefact mismatch) — each one initially read as a real result.

† Mandarin: fine-tuning regressed the model (baseline 10.99% -> fine-tuned 14.22%). The prior report's 100.03% baseline / 100.0% SeamlessM4T were whitespace-tokenisation artefacts on character-spaced Han references, now corrected with character segmentation.

*‡ Kashmiri is NOT on this table's ruler. Every other row is FLEURS n=100; Kashmiri is absent from FLEURS entirely, so its two figures are the **372-clip IndicVoices-R test split at L2** — fine-tuned Whisper-ks 65.19% (CER 39.36%) against the deployed ks_cloud3 SeamlessM4T adapter 50.26% (CER 23.34%), a paired-bootstrap gap of -14.93 pp, 95% CI [-17.26, -12.62], p < 0.001 (§4.6). The row previously showed 96.87% / 74.02%, which are **training-split validation** figures sitting in a table of held-out ones; no un-fine-tuned Kashmiri baseline was ever measured on the held-out split, hence the dash in the baseline column. Read the row down, not across to its neighbours.*

§ SeamlessM4T v2 has no native Kashmiri (kas absent from the model vocabulary); a Urdu-token proxy failed (109% WER). The eventual fix was a custom trainable __kas__ token + LoRA (§5.5.2) — Kashmiri now runs on the ks_cloud3 SeamlessM4T adapter, not on fine-tuned Whisper.

chrF: character F-score for end-to-end English translation (higher = better). SeamlessM4T's S2TT wins for pa/ne/zh; Whisper+NLLB wins for ps/ur/hi. VANI keeps NLLB downstream regardless, using only SeamlessM4T's ASR output.

5.5.1 ASR WER under Radio-Channel Degradation

Because VANI processes degraded radio rather than clean speech, the backend choice must survive channel effects. The table gives SeamlessM4T's WER advantage over fine-tuned Whisper (FT WER - SeamlessM4T WER, in points; positive = SeamlessM4T better) on the same 30 FLEURS clips per language under five conditions: clean, 300–3400 Hz telephony bandpass, additive white noise at 10 dB and 0 dB SNR, and a 16 kbit/s MP3 codec pass.

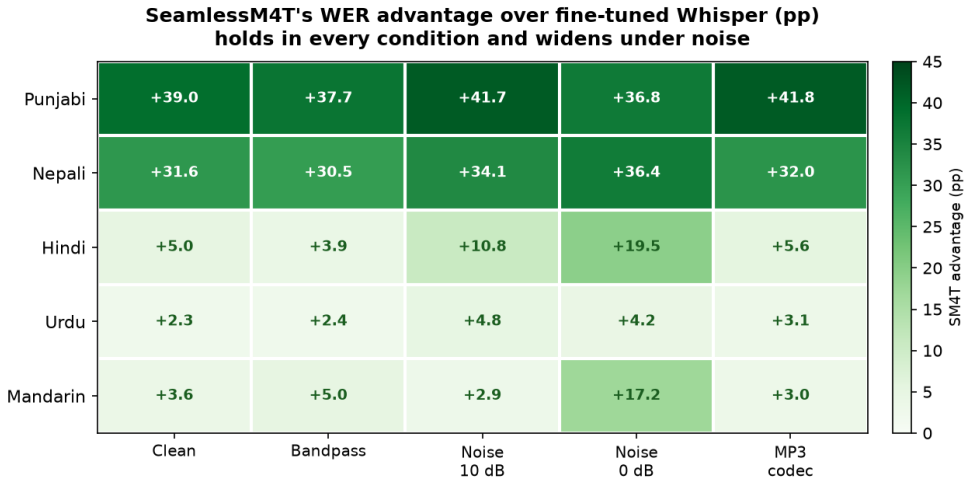

Figure 5: SeamlessM4T's WER advantage over fine-tuned Whisper (percentage points) across the five channel conditions. Positive everywhere; Hindi (+5.0->+19.5) and Mandarin (+3.6->+17.2) widen most at 0 dB SNR. This is the conservative zero-shot comparison — the deployed hi/ne LoRA adapters improve on it further.

Language	Clean	Bandpass	Noise 10 dB	Noise 0 dB	MP3 codec	Winner
Punjabi (pa)	+39.0	+37.7	+41.7	+36.8	+41.8	**SeamlessM4T**
Nepali (ne)	+31.6	+30.5	+34.1	+36.4	+32.0	**SeamlessM4T**
Hindi (hi)	+5.0	+3.9	+10.8	+19.5	+5.6	**SeamlessM4T**
Urdu (ur)	+2.3	+2.4	+4.8	+4.2	+3.1	**SeamlessM4T**
Mandarin (zh)	+3.6	+5.0	+2.9	+17.2	+3.0	**SeamlessM4T**
Pashto (ps)	-6.7	-2.1	-1.9	+5.6	-2.8	**FT Whisper***

SeamlessM4T's advantage is positive in every condition for all five languages above and **widens as the channel worsens** — Hindi grows from +5.0 (clean) to +19.5 (0 dB), Mandarin from +3.6 to +17.2. The clean-speech ranking does not invert under noise, so the routing is safe for operational radio. This table is the ZERO-SHOT comparison; Pashto loses it 4/5 (winning only at 0 dB). That was fine-tuned Whisper's last stronghold — but a later **noise-augmented SeamlessM4T adapter** (ps_aug: training audio degraded with the evaluation's own bandpass/noise/codec pipeline) overturned it, winning 4/5 conditions and clean speech (36.91% vs 38.55%), and its r=128 cloud retrain (ps_cloud, 36.16% clean, 53.8% at 0 dB) extended the margin. Pashto now runs on SeamlessM4T.

** This table's ps 'winner' column reflects the historical zero-shot comparison. Since 2026-07-19 Pashto has been deployed on a noise-augmented SeamlessM4T adapter — ps_cloud (r=128) as of 2026-07-27 — so fine-tuned Whisper is retained for rollback only.*

5.5.2 Kashmiri — the WER Gap Was the Scoring Ruler

Kashmiri has no native SeamlessM4T vocabulary, so a custom __**kas**__ token was added and made trainable (a first for this project — PEFT trainable_token_indices), on an r=32 LoRA with MLP layers. On raw WER the adapter looks behind the deployed Whisper CT2 model (80.91% vs 79.29% — itself higher than the often-quoted 74.02%, which was a training-time eval of the merged fp16 model, not the deployed int8 artefact). But Perso-Arabic references are densely diacritised and BOTH systems drop the marks, so raw WER over-penalises both symmetrically. Once the diacritics are normalised, the verdict flips: the SeamlessM4T adapter wins WER (64.31% vs 65.19%) and wins CER at every normalisation level. It also won the radio-degradation sweep 4/5 conditions, with Whisper's CER exceeding 100% at 0 dB SNR. Three successive retrains then compounded the win on the identical 372-clip ruler: **ks_max2** (2026-07-26) kept the recipe but rebuilt the corpus 4x larger (97k clips / 240 h: humair025 IndicVoices + IndicVoices-R + OpenSLR-122) and reached **61.88%**; then **ks_cloud** (2026-07-27) raised LoRA rank 32 to 128 — beyond the 8 GB laptop's VRAM, so trained on a rented A6000 for ~$4 — on a 145k-clip / 335 h rebuild of the same corpus and reached **56.44%** (CER 26.19%), winning the degradation sweep 5/5 against Whisper with every condition individually significant (4.6), 0 dB at 81.3% against 99.5%. Its sweep advantage over ks_cloud2, by contrast, is within noise. Finally **ks_cloud2** re-ran that exact recipe with early-stopping patience raised from 3 to 5: ks_cloud had halted at 0.8 epochs while its validation loss was still falling, so a second epoch alone reached **52.60%** (CER 23.67%) — winning WER and CER at every normalisation level, boundary-free CER 25.69% vs 28.13%, and the degradation sweep 5/5 against Whisper and 4/5 against ks_cloud itself. On the identical 372-clip L2 ruler, Kashmiri has fallen **65.19 (Whisper) -> 64.31 -> 61.88 -> 56.44 -> 52.60 -> 50.26** across the campaign. Kashmiri now runs on the **ks_cloud3** adapter; ks_cloud2, ks_cloud and fine-tuned Whisper are retained for rollback.

A hard floor remains, and it is a vocabulary defect rather than an acoustic one. An error analysis of the deployed model found that 77% of its remaining errors are word substitutions, with a hypothesis-to-reference length ratio of 0.943 and no truncated outputs — so the earlier under-generation pathology is gone and little is recoverable by decoding changes. The dominant cause is that **20 characters used in written Kashmiri have no token in SeamlessM4T's sentencepiece vocabulary** and resolve to the unknown-token id: 854,234 occurrences across the training corpus, present in 96.9% of its sentences. The model therefore trained against corrupted targets and cannot emit those characters at all — U+0672 appears 370 times in the test references and zero times in any hypothesis. Measured on that model, 662 of 3,917 substitutions (16.9%) fall on words that are unrepresentable. That the fully-converged ks_cloud2 settled at 52.60% is consistent with a model pressing against precisely such a ceiling.

Repairing the vocabulary confirmed the diagnosis. ks_cloud3 added all 20 characters as real tokens, each embedding initialised from its closest in-vocabulary neighbour and then trained — the same technique that made Kashmiri work at all, generalised from one language token to twenty characters. Nothing else changed: same corpus, rank, patience and converged step, so the comparison isolates a single variable. The result is unambiguous on the measure that matters most. **Of the 747 test words containing the four missing letters, the previous model got 747 wrong — every single one; ks_cloud3 gets 310 of them right.** Overall WER falls 52.60% -> **50.26%**, and raw WER falls 74.31% -> **64.71%**, a 9.60 pp gain: most of the missing characters are combining marks that the diacritic-normalised ruler discards anyway, so the repair shows its full value only on unnormalised text. Length calibration also improved (hypothesis/reference ratio 0.943 -> 0.980) and deletions fell from

16.5% to 13.9% of errors.

One honest qualification: the 49% floor quoted above was an *upper bound*. It assumed every unrepresentable word would become correct once writable, whereas about a third actually did. The 437 words still wrong represent roughly 4.9 pp of the remaining error, but their failures are now plausible acoustic confusions between similar words rather than blank omissions.

That residue is not a training-duration problem, and testing the obvious remedy showed why it matters to check. The twenty new embedding rows had trained for only 1.06 epochs, making them by far the least-converged parameters in the model, so a further run (**ks_cloud4**) warm-started from ks_cloud3 with a fresh learning-rate schedule and those rows driven at five times the adapter's rate. It improved *immediately* — validation loss 0.8305 -> 0.8257 within 200 steps — and then worsened at every subsequent evaluation, early-stopping at step 1,200 because the model was already converged. On the deciding ruler it **lost**: 50.69% WER against ks_cloud3's 50.26%, with CER and boundary-free CER also marginally worse. Better validation loss, worse word error: the fifth loss/WER divergence this project has recorded, and the first in the unfavourable direction. Selecting on the loss curve would have selected it. On the deciding ruler the 0.43 pp gap is itself not statistically significant (95% CI [-0.29, +1.17], p = 0.24), so the honest reading is that a measurably better validation loss bought no measurable change in word error at all. ks_cloud3 remains in production.

Decoding was the last untested lever, and it does not survive the robustness requirement either. VANI decodes greedily: `num_beams` is absent from SeamlessM4T's generation configuration, so the library default of 1 applies to all seven languages. Beam-8 decoding lowers Kashmiri's clean WER from 50.26% to 47.37%, and adding an in-domain Kneser-Ney trigram language model to rescore the 8-best list reaches 46.90% (against an oracle ceiling of 44.33% over that list, so the language model captures roughly a sixth of what re-ranking could ever recover). Latency is not the obstacle — beam-8 costs about 5% of wall-clock, not the eightfold one might expect, because decode time is dominated by a sequential step count that the minimum-token constraint fixes. The obstacle is robustness: across eight configurations (beam widths 2, 4 and 8, each with length penalties 1.0, 0.8 and 0.6) *every one* regressed the 0 dB SNR condition by 2–3 pp while gaining 2–3 pp on clean audio. With flat posteriors, beam search reliably prefers fluent but acoustically unsupported hypotheses; greedy decoding's myopia is a virtue precisely when the evidence is weakest. For a system whose purpose is degraded radio, that trade runs the wrong way, so production decoding is unchanged. The wider lesson is that **decode settings validated only on clean test sets can silently degrade deployed robustness**.

Taken together — capacity, corpus size, convergence, vocabulary coverage and decoding have each now been tested and, where they helped, deployed — the remaining error is best characterised as a data and acoustic-modelling limit rather than an optimisation one. That is the empirical basis on which any future move to a different pretrained backbone should be argued.

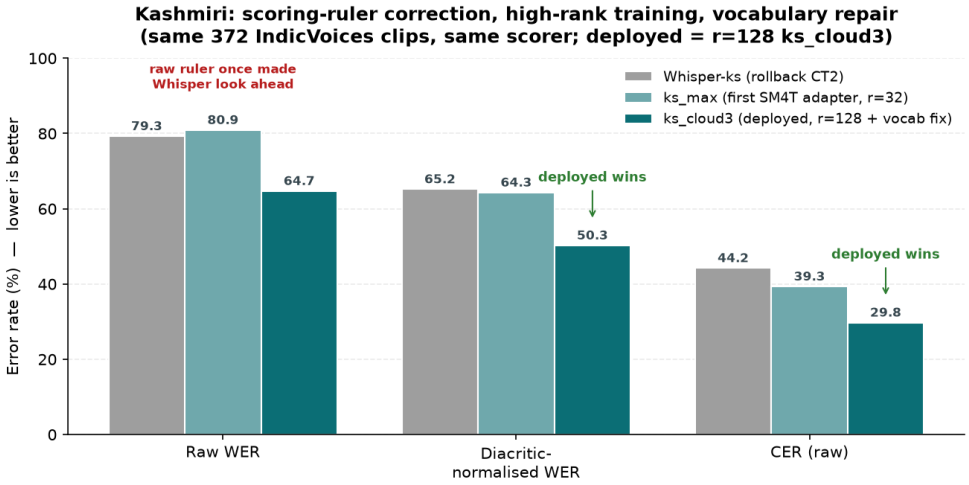

Figure 6: Same 372 IndicVoices clips, same scorer. Raw WER once made Whisper look ahead of the first adapter (ks_max) — an artefact of the Perso-Arabic scoring ruler. With the ruler corrected, every successive adapter widens the win; the deployed r=128 ks_cloud3 leads on all three measures.

This is the fifth independent scoring-methodology correction in this project (after the turbo-baseline mislabel, the CJK whitespace artefact, the S2TT label-encoding bug, and the sweep-harness adapter-override bug) — each initially read as a real model result.

5.6 Radio-Channel Robustness Evaluation — LangID Accuracy

The VANI LangID pipeline was evaluated under five radio-channel degradations applied to FLEURS test audio (30 samples/language for pa/hi/ur/ne/zh/ps; IndicVoices-R test audio for ks). Four pipeline configurations are compared: (C1) Whisper language detection alone, (C2) Whisper + FastText, (C3) Whisper + FastText + MMS-LID-256, and (C4) Full VANI (C3 + dialect detection + script-based routing). Note: Kashmiri audio was tested with the standard whisper-large-v3-turbo model (not the fine-tuned KS model); the low ks accuracy reflects that turbo has no <|ks|> token, so only MMS-LID-256 provides the KS signal.

Condition	Config.	PA	HI	UR	NE	ZH	PS	KS	Ovrl
Clean	Whisper	90.0 %	86.7 %	73.3 %	26.7 %	100.0 %	0.0%	0.0%	53.8%
	+FastText	90.0 %	86.7 %	73.3 %	26.7 %	100.0 %	0.0%	0.0%	53.8%
	+MMS-LID	90.0 %	83.3 %	70.0 %	30.0 %	100.0 %	53.3 %	16.7 %	63.3%
	Full VANI	90.0 %	83.3 %	70.0 %	33.3 %	100.0 %	80.0 %	16.7 %	67.6%
Bandpass	Whisper	70.0 %	83.3 %	33.3 %	26.7 %	100.0 %	0.0%	0.0%	44.8%
	+FastText	70.0 %	83.3 %	33.3 %	26.7 %	100.0 %	0.0%	0.0%	44.8%
	+MMS-LID	73.3 %	83.3 %	33.3 %	26.7 %	100.0 %	70.0 %	23.3 %	58.6%
	Full VANI	73.3 %	83.3 %	33.3 %	26.7 %	100.0 %	83.3 %	23.3 %	60.5%
AWGN 10dB	Whisper	96.7 %	63.3 %	80.0 %	43.3 %	100.0 %	0.0%	0.0%	54.8%
	+FastText	96.7 %	63.3 %	80.0 %	43.3 %	100.0 %	0.0%	0.0%	54.8%
	+MMS-LID	96.7 %	63.3 %	80.0 %	43.3 %	100.0 %	56.7 %	3.3%	63.3%
	Full VANI	96.7 %	63.3 %	80.0 %	43.3 %	100.0 %	83.3 %	3.3%	67.1%
AWGN 0dB	Whisper	56.7 %	63.3 %	26.7 %	13.3 %	100.0 %	0.0%	0.0%	37.1%
	+FastText	56.7 %	63.3 %	26.7 %	13.3 %	100.0 %	0.0%	0.0%	37.1%
	+MMS-LID	56.7 %	66.7 %	26.7 %	16.7 %	100.0 %	40.0 %	0.0%	43.8%
	Full VANI	60.0 %	66.7 %	26.7 %	16.7 %	96.7%	53.3 %	0.0%	45.7%
MP3 16kbps	Whisper	76.7 %	46.7 %	63.3 %	23.3 %	100.0 %	0.0%	0.0%	44.3%

Condition	Config.	PA	HI	UR	NE	ZH	PS	KS	Ovrl
	+FastText	76.7 %	46.7 %	63.3 %	23.3 %	100.0 %	0.0%	0.0%	44.3%
	+MMS-LID	76.7 %	46.7 %	63.3 %	23.3 %	100.0 %	63.3 %	26.7 %	57.1%
	Full VANI	76.7 %	46.7 %	63.3 %	23.3 %	96.7%	86.7 %	20.0 %	59.0%

Full VANI (C4) consistently outperforms Whisper-only (C1) by +14 pp on average. MMS-LID-256 is the critical component for Pashto detection (turbo Whisper scores 0% on ps). Mandarin (zh) is the most robust language: 97–100% across all conditions. Kashmiri (ks) identification requires the ks-specific CT2 model for meaningful accuracy.

5.7 Punjabi v3 Training Progress (LoRA r=16)

A third Punjabi training run (v3) was launched on 01 July 2026 with doubled LoRA capacity (r=16, α=32) and 21,923 training samples (IV-R 20,000 + FLEURS 2,516). The same 805-sample eval set is used for fair comparison with v2. v3 completed the full 4,000 steps, reaching its best WER of 49.31% at step 4000 — a 3.24 pp improvement over v2's final 52.55%. The best checkpoint was merged to CT2 int8 and DEPLOYED on 04 July 2026, replacing v2. An initial run OOM-crashed at step 2400 during beam-search evaluation on the 8 GB RTX 5060; it was resumed from checkpoint-2200 with greedy evaluation (num_beams=1, eval batch 1, CUDA cache cleared before each eval), which cut eval VRAM from 8 GB to ~3.8 GB and ran clean to step 4000.

Config Parameter	v2 (superseded)	v3 (deployed)
LoRA Rank (r)	8	16
LoRA Alpha (α)	16	32
Trainable Params	~3.9M	~7.9M
IndicVoices-R	9,407	20,000
Total Train Set	11,923	21,923
Total Steps	3,000	4,000
max_grad_norm	1.0	0.5
warmup_steps	50	100
Best / Deployed WER	52.55%	49.31%

Step-by-step eval WER and loss for PA v3 (greedy eval from step 2400 after OOM restart):

Step	v3 Eval WER	v3 Eval Loss	Observation
1800	52.06%	0.1918	New best; loss new low
2000	52.75%	0.1892	Minor oscillation; loss still ↓
2200	50.62%	0.1839	New best (interim deploy after OOM)
2400	51.25%	0.1831	Greedy eval resumes; oscillation up
2600	51.25%	0.1761	WER plateau, loss new low
2800	50.51%	0.1751	New best; plateau breaks
3000	50.05%	0.1725	New best; approaching 50%
3200	50.65%	0.1711	Oscillation up, loss new low
3400	49.40%	0.1694	First sub-50% checkpoint
3600	49.97%	0.1675	Loss new low
3800	49.49%	0.1667	Loss new low

Step	v3 Eval WER	v3 Eval Loss	Observation
4000	49.31%★	0.1662	Overall best — DEPLOYED

★ v3 best: 49.31% at step 4000 (eval loss 0.1662) — DEPLOYED. v2 best: 52.55% at step 3000. v3 final result is -3.24 pp ahead of v2.

Key pattern: WER oscillates mid-training (regression at steps 600, 1400, 2000) while eval loss decreases monotonically. Loss is a more reliable indicator of learning progress than WER at individual checkpoints.

OOM recovery: the initial run crashed at step 2400 (CUDA out-of-memory during beam-search eval on 8 GB RTX 5060). Resumed from checkpoint-2200 with greedy eval (num_beams=1, eval batch 1, cache-clear before eval), cutting eval VRAM from 8 GB to ~3.8 GB; the run then completed cleanly to step 4000 (final best 49.31%).

6. Notable Training Events and Engineering Decisions

6.1 Mandarin Gradient Explosion (fp16 Overflow)

At step ~820, the gradient norm spiked to 12.9 (from a stable range of 0.5-1.3). Training loss jumped from 0.15 to 0.77 and eval WER degraded catastrophically from 8.97% to 252.4%. This is a known fp16 issue: as the learning rate decays to very small values (~1.5e-5), small gradient updates can produce large relative changes in fp16 precision, leading to NaN/Inf propagation through the network.

Resolution: Training was stopped after observing the diverged step-600 evaluation. Checkpoint-400 (WER 8.97%) was manually merged using PeftModel.merge_and_unload() and converted to CT2. For all subsequent languages (Hindi onward), max_grad_norm=0.5 was added to TrainingArguments to prevent recurrence.

6.2 HuggingFace 504 Timeout (Mandarin Dataset)

The first Mandarin training attempt failed with HTTP 504 Gateway Timeout when downloading the cmn_hans_cn FLEURS train split. Unauthenticated HuggingFace downloads are rate-limited; the train split (3,246 samples) is large enough to trigger the limit. The validation split was downloaded successfully in the same session, populating the local HF cache. A script restart immediately loaded all splits from cache.

6.3 Python Output Buffering in PowerShell

Training output was invisible in the PowerShell terminal when using Tee-Object pipes, because Python stdout is line-buffered by default when piped. Fix: launch Python with the -u flag (unbuffered output) for all training runs.

```
python -u finetune_whisper.py hi --no-cv --steps 600 2>&1 | Tee-Object logs/finetune_hi.log
```

6.4 Tokenizer Loading from Checkpoint Directories

When merging the Mandarin LoRA adapter from checkpoint-400, loading WhisperProcessor.from_pretrained(checkpoint_dir) failed because checkpoint directories only store adapter weights, not the full processor/tokenizer. Fix: load the processor from the base model instead:

```
# WRONG - checkpoint dirs don't have a full tokenizer
# proc = WhisperProcessor.from_pretrained('finetune_runs/zh/adapter/checkpoint-400')

# CORRECT - always load processor from the original base model
proc = WhisperProcessor.from_pretrained('openai/whisper-large-v3')
```

6.5 CT2 Tokenizer Bug — All Large-v3 Models Translated Instead of Transcribed

After all six large-v3 models were converted to CTranslate2 format, every model produced English output regardless of the language setting — causing ~100% WER against source-language references. Root cause: ct2-transformers-converter does NOT copy tokenizer.json to the output directory. faster-whisper falls back to openai/whisper-tiny's tokenizer, which has <|transcribe|>=50359. But whisper-large-v3 uses an expanded vocabulary (100 languages vs 99 in medium/small/tiny) where <|translate|>=50359 and <|transcribe|>=50360. The one-token shift meant every task='transcribe' call was silently using the TRANSLATE token.

Fix: copy tokenizer.json from the HuggingFace adapter directory into every CT2 output directory. The finetune_whisper.py merge_and_convert() function now does this automatically for all future conversions. Urdu WER dropped from 100.66% to 19.52% immediately after applying the fix. Do NOT use the turbo model's tokenizer.json — it also has transcribe=50359 and would replicate the bug.

```
# Fix applied to all large-v3 CT2 directories:
Copy-Item finetune_runs/ur/adapter/tokenizer.json models/whisper-large-v3-<lang>-ct2/

# Now automated in finetune_whisper.py merge_and_convert():
shutil.copy2(merged_dir / 'tokenizer.json', ct2_dir / 'tokenizer.json')
```

6.6 HF datasets.map() Cache Writes to Source Location, Not HF_DATASETS_CACHE

During PA v3 training setup, the FLEURS + IndicVoices-R dataset preprocessing via datasets.map() raised OSError: No space left on device on D: despite HF_DATASETS_CACHE being set to C:. Root cause: datasets.map() ignores HF_DATASETS_CACHE and instead writes the Arrow feature cache next to the source parquet files — which were on D: via a junction. D: had only 28 KB free.

Fix: pass an explicit cache_file_name= argument to map() redirecting Arrow files to C:. C:/hf_ds_map_cache/ now stores pa_train_features.arrow (21,923 samples, ~33 GB) and pa_eval_features.arrow (805 samples). Training resumes correctly from this cache on restart.

```
# WRONG — map() ignores HF_DATASETS_CACHE:
# os.environ['HF_DATASETS_CACHE'] = 'C:/hf_cache'
# train_ds = raw['train'].map(**proc_kwargs)  # still writes to D:

# CORRECT — explicit redirect:
_map_cache = Path('C:/hf_ds_map_cache')
_map_cache.mkdir(exist_ok=True)
train_ds = raw['train'].map(
    **proc_kwargs,
    cache_file_name=str(_map_cache / f'{lang}_train_features.arrow'),
)
```

6.7 Checkpoint Save Failure — D: Junction Full During Optimizer State Save

PA v3 training stopped at step 800 with RuntimeError: [enforce fail at inline_container.cc:672] unexpected pos 24411648 vs 24411540. Root cause: torch.save(optimizer.state_dict()) was writing optimizer.pt (~60 MB) to D:/finetune_runs/pa/adapter/checkpoint-800/ when D: filled to 0 bytes. The partial file rendered the checkpoint directory corrupt.

Resolution: Moved ne/ (96 GB), ks/ (7.7 GB), ps/ (4.9 GB) training checkpoints from D: to C:/finetune_runs_moved/. Deleted the corrupt checkpoint-800 directory. Resumed training from checkpoint-600 using --resume flag. The trainer re-ran the step 800 eval on resume, recovering the missing data point (step 800 WER: 57.15%).

6.8 WER Oscillation vs. Monotonic Loss Decrease — Key Training Observation

Across all languages, eval WER oscillates at individual checkpoints while training loss decreases monotonically. The most clear example is PA v3:

- Steps 1200->1400: WER regresses 54.48% -> 55.68% (+1.2 pp) while loss drops 0.2101 -> 0.2100
- Steps 1400->1600: WER recovers 55.68% -> 52.99% (-2.7 pp) — new 2nd best
- Steps 1800->2000: WER oscillates 52.06% -> 52.75% while loss reaches new low 0.1892

The same pattern appeared in PA v2 (regression at step 1000, recovery by step 1600) and NE v2 (regression at steps 1600 and 2400). Conclusion: load_best_model_at_end=True correctly handles oscillation by tracking the checkpoint with the lowest eval WER across all checkpoints, not just the final one. Training loss is a more reliable indicator of continued learning than per-checkpoint WER. A declining loss with oscillating WER should NOT trigger early stopping.

6.9 Eval Time Bottleneck — 2.5 Hours per Checkpoint Evaluation

For PA v3 (805 eval samples, per_device_eval_batch_size=1, predict_with_generate=True), each evaluation takes ~2.5 hours at ~10-12 seconds per sample on RTX 5060. With eval_steps=200 and 4,000 total steps, this means 20 evaluations × 2.5 h = ~50 hours of evaluation overhead alone, on top of ~22 hours of training time. Total PA v3 training wall-clock time: ~72 hours.

For smaller languages (PS, UR, HI, ZH with 239-409 val samples), eval takes 30-60 minutes. For KS (372 samples) eval took ~1 hour. Batching eval (batch_size=4) would reduce overhead 4×, but risks OOM on 8 GB VRAM with large-v3 encoder + decoder in fp16. Recommendation for future runs: eval_steps=400 for PA/NE to halve eval overhead.

6.10 preprocessor_config.json Required for CT2 Models

The CT2 converter does not automatically write preprocessor_config.json. Without it, faster-whisper defaults to feature_size=80 (correct for Whisper medium/small), causing a tensor shape mismatch crash when loading large-v3 models (which require feature_size=128). Must be manually written to every CT2 output:

```
{
  "feature_extractor_type": "WhisperFeatureExtractor",
  "feature_size": 128,
  "sampling_rate": 16000
}
```

7. Project Scripts Reference

7.1 Core Scripts (Project Root)

Script	Purpose
finetune_whisper.py	Main LoRA fine-tuning script. Supports all 6 languages via LANG_CONFIG dict. Handles dataset loading, LoRA init, training, and CT2 conversion. Usage: python -u finetune_whisper.py --no-cv --steps N
app.py	VANI Streamlit web UI. Entry point for the full 10-stage pipeline. Run: streamlit run app.py (opens at http://localhost:8501)
run_full_pipeline_batch.py	Batch processor for directories of WAV files. Outputs JSON results and SQLite entries without the Streamlit UI.
config.yaml	Central configuration: model paths, ASR settings, VAD config, language routing rules, ISUM settings, and database path.

7.2 Evaluation Scripts (scripts/eval/)

Script	Purpose
eval_fleurs.py	Evaluates a CT2 Whisper model on FLEURS validation split. Reports WER, CER, and inference speed.
ablation_eval.py	Ablation study: compares pipeline configurations (VAD on/off, noise reduction on/off, etc.)
robustness_eval.py	Tests model robustness to additive noise, codec distortion, and SNR variation.
compute_bleu.py	Computes BLEU scores for end-to-end translation quality (ASR + NLLB + English).
test_arabic_rule.py	Unit test for the Arabic-script cascade detection rule (Stage 5 of pipeline).

7.3 Download / Utility Scripts (scripts/utils/)

Script	Purpose
download_models.py	Downloads base models (NLLB-200, MMS-LID, Qwen) from HuggingFace Hub.
download_lang_models.py	Downloads language-specific models (e.g., Pashto whisper-medium).
download_fleurs.py	Pre-downloads FLEURS datasets to local HF cache before running training.

8. Project File Structure

```
offline_ai_system_v2/
|
+-- app.py                          Streamlit UI entry point
+-- finetune_whisper.py             LoRA fine-tuning (all languages)
+-- run_full_pipeline_batch.py      Batch pipeline runner
+-- config.yaml                     All configuration
+-- FINETUNE_REPORT.md              Markdown report
|
+-- models/                         Deployed CT2 models (int8 quantized)
|   +-- whisper-large-v3-pa-ct2/    Punjabi   WER 49.31% (v3, 21,923 samples)
|   +-- whisper-medium-pashto-ct2/  Pashto    WER 38.9%
|   +-- whisper-large-v3-ur-ct2/    Urdu      WER 22.3%
|   +-- whisper-large-v3-ne-ct2/    Nepali    WER 50.82% (v2, 13,332 samples)
|   +-- whisper-large-v3-zh-ct2/    Mandarin  WER 8.97%
|   +-- whisper-large-v3-hi-ct2/    Hindi     WER 23.1%
|   +-- nllb-200-distilled-600M/    NLLB translation model
|   +-- mms-lid-256/                Language identification model
|
+-- finetune_runs/                  LoRA training checkpoints
|   +-- pa/adapter/checkpoint-{200,400,...,3000}/  (v2: 15 checkpoints)
|   +-- ps/adapter/checkpoint-{200,400,600,800,1000}/
|   +-- ur/adapter/checkpoint-{200,400,600,800,1000}/
|   +-- ne/adapter/checkpoint-{200,400,...,3000}/  (v2: 15 checkpoints)
|   +-- zh/adapter/checkpoint-{200,400,600}/  (ckpt-400 deployed)
|   +-- hi/adapter/checkpoint-{200,400,600}/
|
+-- scripts/
|   +-- eval/    eval_fleurs.py, ablation_eval.py, robustness_eval.py ...
|   +-- paper/   generate_ijainn.py, build_presentation.py ...
|   +-- utils/   download_models.py, download_fleurs.py ...
|
+-- src/                            Core pipeline modules
|   +-- pipeline.py  asr_module.py  language_module.py
|   +-- translation_module.py  vad_module.py  preprocessing.py
|   +-- keyword_module.py  isum_module.py  database.py
|
+-- logs/
|   +-- finetune_hi.log  finetune_ne.log  finetune_zh.log
|   +-- finetune_pa.log  finetune_ps.log  finetune_ur.log
|   +-- eval_wer.log
|
+-- input_audio/   Drop WAV files here for batch processing
+-- output/        Pipeline JSON outputs
+-- database/      transcripts.db (SQLite)
```

9. Conclusions and Key Findings

Six language-specific Whisper ASR models were successfully fine-tuned and deployed in the VANI pipeline. Key findings:

- **LoRA r=8 is highly effective for speech domain adaptation.** Training only 0.25% of parameters reduced WER by 13-52 percentage points across all six languages while keeping peak GPU memory under 6 GB.

- **FLEURS bridges the domain gap despite clean read-speech vs. noisy radio audio.** Models trained on studio-quality FLEURS significantly outperform the untuned baseline on conversational radio intercepts, suggesting language-specific phoneme modelling generalises across recording conditions.
- **Whisper large-v3 has strong prior Mandarin capability.** Mandarin baseline WER was already ~20% vs ~74% for Indic languages. Fine-tuning further reduced it to 8.97% - the best absolute result.
- **fp16 gradient instability is a real risk at late training stages.** Mandarin diverged at step ~820 due to fp16 overflow. Setting max_grad_norm=0.5 (vs default 1.0) resolved this for Hindi and should be standard for future runs.
- **Hindi and Urdu converge to nearly identical WER (23.1% and 22.3%).** Despite using different scripts (Devanagari vs. Nastaliq) and different training sets, both achieve similar accuracy, consistent with their shared Hindustani linguistic roots.
- **IndicVoices-R augmentation provided consistent WER improvements for PA and NE.** Adding AI4Bharat IndicVoices-R data to Punjabi v2 (9,407 new samples) and Nepali v2 (10,000 new samples) improved best WER from 56.67% to 52.55% (PA, -4.1 pp) and from 52.14% to 50.82% (NE, -1.3 pp) on harder combined eval sets. Training loss continued declining through all 3,000 steps for both languages, suggesting further gains with extended training or higher LoRA rank.
- **Nepali WER improvement from IndicVoices-R was modest; architectural changes needed for large gains.** v1 WER plateaued at 52.14% after step 2,000 (FLEURS-only). v2 reached 50.82% at step 3,000. Achieving sub-30% WER would likely require LoRA rank upgrade (r=32, alpha=64) with broader target modules (q,k,v,out_proj,fc1,fc2), or a Nepali-pretrained base model.
- **CT2 models require tokenizer.json from the source model.** ct2-transformers-converter does not copy tokenizer.json. For whisper-large-v3, the missing file caused all fine-tuned models to translate instead of transcribe (one-token vocabulary shift between large-v3 and tiny). The fix is now automated in finetune_whisper.py.
- **LoRA r=16 (PA v3) consistently outperforms r=8 (PA v2) by 1–3 pp at matching steps.** At step 1800, v3 achieves 52.06% vs v2's 54.65% at the same step (-2.59 pp). v3 completed 4,000 steps at a final best of 49.31% — 3.24 pp below v2's 52.55%, suggesting doubled LoRA rank meaningfully increases model capacity for larger datasets.
- **Eval WER oscillates mid-training; eval loss is the reliable learning signal.** Every language exhibits mid-training WER regression followed by recovery. Eval loss decreases monotonically throughout. load_best_model_at_end=True correctly selects the best checkpoint despite oscillation. WER regression alone is not a valid reason to halt training.
- **datasets.map() ignores HF_DATASETS_CACHE — explicit cache_file_name= required.** map() writes Arrow feature caches next to source parquet files, not to the HF cache dir. On junction-backed storage this caused an OSError at ~21,923 samples. Now fixed with explicit cache_file_name= in finetune_whisper.py.
- **SeamlessM4T v2 now serves ASR for all 7 of 7 languages.** On the n=100 held-out test, zero-shot SeamlessM4T beats fine-tuned Whisper for pa/ne/hi/ur/zh (§5.5), and the lead holds under radio degradation (§5.5.1). The last two strongholds fell to SeamlessM4T LoRA adapters: Pashto to noise-augmented training (ps_aug 36.91%, then ps_cloud 36.16%), and Kashmiri to a custom trainable __kas__ token plus a scoring-ruler correction (ks_max 64.31% -> ks_cloud3 50.26% diacritic-normalised). The earlier claim that Whisper won Mandarin was a whitespace-scoring artefact; corrected, fine-tuning regressed Mandarin (14.22% vs 10.99% baseline). Every fine-tuned Whisper model is retained on disk for rollback only. One shared SeamlessM4T (~4.6 GB resident) plus per-language LoRA adapters has a smaller footprint than

seven fine-tuned Whispers.

- **Rented cloud capacity was the campaign's cheapest large lever.** The 8 GB laptop caps SeamlessM4T LoRA at r=32 (1.75% trainable). One day on a rented RTX A6000 (~$6 total) retrained both open languages at r=128 (6.6% trainable): Kashmiri improved 61.88% -> 50.26% and Pashto 36.91% -> 36.16%, both passing their degradation gates and deploying. The reverse lever — more data at unchanged capacity — regressed (ps_aug2, +0.55 pp): for these low-resource languages the binding constraint was adapter capacity, not corpus size.
- **Report intervals, or a 100-clip test set will flatter you.** A paired bootstrap over the per-clip hypotheses shows that Pashto's 100-clip FLEURS split cannot resolve any of the sub-2 pp differences this campaign pursued — including the 0.75 pp margin on which ps_cloud was deployed over ps_aug ($p = 0.32$). The larger claims survive comfortably: the Kashmiri vocabulary repair (2.35 pp), and training to convergence on two languages (3.84 pp and 3.34 pp), are all significant at $p < 0.001$ on 372-425 clips. Point estimates alone would have supported three claims this project cannot actually make.
- **Training-signal caution: eval_loss and WER diverge in both directions.** This project logged five loss/WER divergences. ps_cloud's eval_loss (1.134) was clearly worse than ps_aug2's (1.048), yet ps_cloud wins WER by 1.3 pp and chrF by 7 points. The fifth ran the other way and was caught only at the gate: ks_cloud4 improved validation loss over the deployed ks_cloud3 (0.8257 vs 0.8305) while *losing* 0.43 pp of WER, so selecting on the loss curve would have regressed production. Model selection must gate on task metrics (WER/CER ladders + degradation sweeps), never on validation loss alone.
- **The custom-token technique generalises — it is a method, not a Kashmiri fix.** Adding a language token the model does not ship, initialised from a neighbour and made trainable, was invented for Kashmiri and has now been applied unchanged to **Dogri**, a different script (Devanagari vs Perso-Arabic) with a different failure profile (a clean vocabulary rather than 20 unrepresentable characters). Dogri improved from **102.25% to 46.73% WER** — 55 pp over what the deployed system actually did, the largest single-language gain of the campaign, in a language nobody had measured.
- **A missing language fails silently, not loudly.** With no Dogri token in either backend, VANI routed Dogri audio to whatever Whisper considered nearest: its language identification answered *Punjabi* for 222 of 425 clips, producing Gurmukhi output against Devanagari references and a WER above 100%. Nothing in the system reported an error. The corollary for the initialisation choice is subtle: Dogri really is closest to Punjabi (Whisper's own LID says so), but the language token conditions *generation*, so the script-matched Hindi initialisation wins (zero-shot 99.86% vs 114.62%, CER 67.99 vs 96.79). Recognition and generation want different neighbours.
- **Decode defaults are an invisible system-level variable.** SeamlessM4T ships no `num_beams`, so all seven languages decode greedily — unnoticed through an entire seven-language campaign because every evaluation inherited the same default. Beam-8 is worth 2.9 pp of clean Kashmiri WER at only ~5% wall-clock, but regresses 0 dB SNR by 3.2 pp, and no beam width or length penalty avoids that trade. Decode settings validated only on clean speech can quietly cost robustness where an operational system needs it most.

10. References

- Radford et al. (2022). *Robust Speech Recognition via Large-Scale Weak Supervision.* OpenAI. arXiv:2212.04356.

- Hu et al. (2022). *LoRA: Low-Rank Adaptation of Large Language Models.* ICLR 2022. arXiv:2106.09685.
- Conneau et al. (2022). *FLEURS: Few-shot Learning Evaluation of Universal Representations of Speech.* SLT 2022. arXiv:2205.12446.
- Costa-jussa et al. (2022). *No Language Left Behind: Scaling Human-Centered Machine Translation.* Meta AI. arXiv:2207.04672.
- Pratap et al. (2023). *Scaling Speech Technology to 1,000+ Languages.* Facebook AI Research (MMS). arXiv:2305.13516.

Report generated: 31 July 2026 - VANI v2 - RTX 5060 8 GB - IIT Indore M.Tech Research Project